

داده کاوی

دانشگاه آزاد شیراز - دانشکده مهندسی کامپیوتر

ترم اول سال تحصیلی ۱۴۰۴-۱۴۰۵

فصل دوم: Data, measurements, and data preprocessing

استاد: دکتر امین اسکندری

تیم طراحی محتوای آموزشی و ویراستاری (علمی و ادبی): حمید نامجو و امیرحسین همتی

فهرست

- 2.1 Data types
 - 2.1.1 Nominal attributes
 - 2.1.2 Binary attributes
 - 2.1.3 Ordinal attributes
 - 2.1.4 Numeric attributes
 - 2.1.5 Discrete vs. continuous attributes
- 2.2 Statistics of data
 - 2.2.1 Measuring the central tendency
 - 2.2.2 Measuring the dispersion of data
 - 2.2.3 Covariance and correlation analysis
 - 2.2.4 Graphic displays of basic statistics of data
- 2.3 Similarity and distance measures
 - 2.3.1 Datamatrix vs. dissimilaritymatrix
 - 2.3.2 Proximity measures for nominal attributes
 - 2.3.3 Proximity measures for binary attributes
 - 2.3.4 Dissimilarity of numeric data: Minkowski distance
 - 2.3.5 Proximity measures for ordinal attributes
 - 2.3.6 Dissimilarity for attributes of mixed types
 - 2.3.7 Cosine similarity
 - 2.3.8 Measuring similar distributions: the Kullback-Leibler divergence
 - 2.3.9 Capturing hidden semantics in similarity measures
- 2.4 Data quality, data cleaning, and data integration
 - 2.4.1 Data quality measures
 - 2.4.2 Data cleaning
 - 2.4.3 Data integration
- 2.5 Data transformation
 - 2.5.1 Normalization
 - 2.5.2 Discretization
 - 2.5.3 Data compression
 - 2.5.4 Sampling
- 2.6 Dimensionality reduction
 - 2.6.1 Principal components analysis
 - 2.6.2 Attribute subset selection
 - 2.6.3 Nonlinear dimensionality reduction methods

Section 2.1 (Data Types)

2.1.1. Nominal attributes

ویژگی‌های اسمی

ویژگی‌های اسمی (Nominal Attributes) متغیرهای کیفی هستند که نمایانگر دسته‌ها یا برچسب‌های متمایز با هیچ‌گونه ترتیب یا رتبه‌بندی ذاتی بین آن‌ها می‌باشند. این ویژگی‌ها داده‌ها را بر اساس تفاوت‌های کیفی به گروه‌های مختلف طبقه‌بندی می‌کنند. به عنوان مثال، رنگ‌ها، جنسیت، انواع حیوانات و موارد مشابه، همگی از ویژگی‌های اسمی به شمار می‌روند.

کد پایتون کاربردی برای ویژگی‌های اسمی

بیاپید یک قطعه کد پایتون ساده با استفاده از کتابخانه **pandas** برای کار با یک مجموعه داده حاوی ویژگی‌های اسمی ایجاد کنیم. ما یک مجموعه داده نمونه را بارگذاری کرده، ویژگی‌های اسمی را کاوش می‌کنیم و عملیات **One-Hot Encoding** را انجام می‌دهیم.

In [2]: **import** pandas **as** pd

```
# رو وارد می‌کنیم 'pandas' این قسمت اول کده، ما به کتابخونه به اسم
# 'pandas' به ابزار خیلی پرکاربرده برای کار کردن با داده‌ها، مخصوصاً وقتی داده‌ها توی یه جدول مرتب باشن
# ما از این کتابخونه استفاده می‌کنیم که داده‌ها رو مثل یه جدول اکسل بخونیم و دستکاریشون کنیم

# حاوی داده‌های نمونه ماست (dictionary) این دیکشنری
# 'Color' یا 'Animal' ما این داده‌ها رو به صورت ستون به ستون تعریف کردیم، مثلاً ستون
# هستن؛ یعنی فقط اسم یا دسته هستن و هیچ ترتیبی بینشون وجود نداره (Nominal) این داده‌ها از نوع اسمی
# بهتر یا بدتر نیست، فقط یه دسته متفاوت 'Cat' از 'Dog' مثلاً
data = {
    'Animal': ['Dog', 'Cat', 'Fish', 'Bird', 'Snake'],
    'Color': ['Brown', 'Black', 'Gold', 'Blue', 'Green'],
    'Habitat': ['Forest', 'Home', 'Aquarium', 'Sky', 'Jungle']
}

# تبدیل می‌کنیم 'DataFrame' این داده‌ها رو به یه 'pandas' حالا با استفاده از
# باهاش کار می‌کنه و داده‌ها رو به شکل منظم نشون می‌ده 'pandas' همون جدولیه که
df = pd.DataFrame(data)

# اصلی رو چاپ می‌کنیم 'DataFrame' یه تابع ساده‌اس که هر چیزی رو بهمون نشون می‌ده. اینجا 'print'
# تا ببینیم داده‌های خام ما قبل از هر پردازشی چه شکلی هستن
print("دیتاست اصلی:")
print(df)
print("\n")

# این یه لیست از ستون‌های اسمیه که می‌خوایم بعداً روشن عملیات انجام بدیم
nominal_attributes = ['Animal', 'Color', 'Habitat']

# ساده‌اس که روی تک‌تک ستون‌های اسمی که تو لیست بالا گذاشتیم، می‌چرخه (for Loop) این یه حلقه
# هر ستون رو پیدا کنه و بهمون نشون بده (Unique values) استفاده می‌کنیم که مقادیر منحصر به فرد 'unique' داخل حلقه، از تابع
# مختلف داره 'Category' این کار کمک می‌کنه بفهمیم هر ستون چند تا دسته یا
for attribute in nominal_attributes:
    print(f"{attribute}: {df[attribute].unique()}")

# رو اجرا می‌کنه 'One-Hot Encoding' این مهم‌ترین بخش کده و مفهوم
# هست که ستون‌های اسمی رو می‌گیره 'pandas' یه تابع از 'pd.get_dummies'
# و هر کدوم از مقادیر اون ستون رو به یک ستون جدید تبدیل می‌کنه
# تو این ستون‌های جدید، اگه مقدار اصلی وجود داشته باشه، عدد 1 و اگه نداشته باشه، عدد 0 می‌داره
# این روش برای آماده‌سازی داده‌های اسمی برای الگوریتم‌های یادگیری ماشین خیلی مهمه
# چون بیشتر الگوریتم‌ها با اعداد کار می‌کنن، نه با کلمات
df_encoded = pd.get_dummies(df, columns=nominal_attributes)

# انجام شده، چاپ می‌کنیم 'One-Hot Encoding' جدید رو بعد از این که 'DataFrame' اینجا
# می‌تونید ببینید که ستون‌های اسمی قدیمی حذف شدن و ستون‌های عددی جدیدی به جای اون‌ها اضافه شدن
print("\nOne-Hot Encoding دیتاست بعد از")
print(df_encoded)
```

دیتاست اصلی:

	Animal	Color	Habitat
0	Dog	Brown	Forest
1	Cat	Black	Home
2	Fish	Gold	Aquarium
3	Bird	Blue	Sky
4	Snake	Green	Jungle

Animal: ['Dog' 'Cat' 'Fish' 'Bird' 'Snake']

Color: ['Brown' 'Black' 'Gold' 'Blue' 'Green']

Habitat: ['Forest' 'Home' 'Aquarium' 'Sky' 'Jungle']

One-Hot Encoding: دیتاست بعد از

	Animal_Bird	Animal_Cat	Animal_Dog	Animal_Fish	Animal_Snake	\
0	False	False	True	False	False	
1	False	True	False	False	False	
2	False	False	False	True	False	
3	True	False	False	False	False	
4	False	False	False	False	True	

	Color_Black	Color_Blue	Color_Brown	Color_Gold	Color_Green	\
0	False	False	True	False	False	
1	True	False	False	False	False	
2	False	False	False	True	False	
3	False	True	False	False	False	
4	False	False	False	False	True	

	Habitat_Aquarium	Habitat_Forest	Habitat_Home	Habitat_Jungle	Habitat_Sky
0	False	True	False	False	False
1	False	False	True	False	False
2	True	False	False	False	False
3	False	False	False	False	True
4	False	False	False	True	False



توضیح مثال کد پایتون

در این مثال:

- ما یک دیتافریم (DataFrame) پانداس با سه ویژگی اسمی: '**Animal**' (حیوان)، '**Color**' (رنگ) و '**Habitat**' (زیستگاه) ایجاد می‌کنیم.
- ما مقادیر منحصر به فرد (Unique Values) را در هر ویژگی اسمی کاوش می‌کنیم.
- ما از تابع **pd.get_dummies()** برای انجام **one-hot encoding** استفاده می‌کنیم و هر ویژگی اسمی را به ستون‌های دودویی (Binary) تبدیل می‌کنیم.
- دیتافریم نهایی (**df_encoded**) پس از عملیات **one-hot encoding** نمایش داده می‌شود.

2.1.2. Binary attributes

0

1

ویژگی‌های دودویی

ویژگی‌های دودویی (Binary Attributes) متغیرهای کیفی هستند که می‌توانند یکی از دو مقدار ممکن را بپذیرند و معمولاً

نشان‌دهنده حضور یا عدم حضور یک ویژگی خاص می‌باشند. این ویژگی‌ها در بسیاری از مجموعه داده‌ها از اهمیت بالایی

برخوردارند و نمونه‌هایی از آن‌ها شامل **Yes/No** (بله/خیر)، **True/False** (درست/نادرست)، **1/0** یا هر دو دسته متمایز دیگر

می‌شوند.



کد پایتون کاربردی برای ویژگی‌های دودویی

بیااید یک قطعه کد پایتون با استفاده از کتابخانه **pandas** برای کار با یک مجموعه داده حاوی ویژگی‌های دودویی ایجاد کنیم. ما

یک مجموعه داده نمونه را بارگذاری کرده، ویژگی‌های دودویی را کاوش می‌کنیم و عملیات‌های پایه را روی آن‌ها انجام می‌دهیم.

In [3]: **import** pandas **as** pd

```
# رو وارد می‌کنیم 'pandas' این قسمت اول کده. مثل قبل، کتابخونه #
# ابزار اصلی ما برای کار کردن با داده‌ها و جدول‌هاست 'pandas' #
# ما از این کتابخونه استفاده می‌کنیم که داده‌ها رو مثل یه جدول اکسل بخونیم و دستکاریشون کنیم #

# داده‌های نمونه رو در خودش نگه داشته (dictionary) این دیکشنری #
```

هستن. یعنی فقط دو مقدار ممکن دارن، مثل ۱ و ۰ (Binary) این بار، داده‌های ما از نوع باینری # نشون می‌ده که دانشجو امتحان رو قبول شده (۱) یا نه (۰) 'PassedExam' اینجا # هم نشون می‌ده که دانشجو تو اون درس ثبت‌نام کرده (۱) یا نه (۰) 'EnrolledInCourse'

```
data = {
    'StudentID': [1, 2, 3, 4, 5],
    'PassedExam': [1, 0, 1, 1, 0],
    'EnrolledInCourse': [1, 1, 0, 1, 0]
}
```

تبدیل می‌کنیم 'DataFrame' این داده‌ها رو به 'pandas' حالا با استفاده از # باهاش کار می‌کنه و داده‌ها رو به شکل منظم نشون می‌ده 'pandas' همون جدولیه که 'DataFrame' df = pd.DataFrame(data)

اصلی رو چاپ می‌کنیم 'DataFrame' به تابع ساده‌اس که هر چیزی رو بهمون نشون می‌ده. اینجا 'print' # تا ببینیم داده‌های خام ما قبل از هر پردازشی چه شکلی هستن # print("دیتاست اصلی:") print(df) print("\n")

اینجا به لیست از ستون‌های باینری رو تعریف می‌کنیم. binary_attributes = ['PassedExam', 'EnrolledInCourse']

این یه حلقه ساده‌اس که روی ستون‌های باینری می‌چرخه # رو صدا می‌زنیم که تعداد تکرار هر مقدار رو تو اون ستون بشماره 'value_counts()' تابع # مثلاً بهمون می‌گه چند تا دانشجو امتحان رو قبول شدن (۱) و چند تا رد (۰) # این کار برای فهمیدن توزیع داده‌ها خیلی مفیده # for attribute in binary_attributes: print(f"تعداد هر مقدار برای {attribute}: \n{df[attribute].value_counts()}\n")

اینجا به عملیات ساده ریاضی روی ستون‌های باینری انجام می‌دیم # به ستون جدید می‌سازه که جمع مقادیر دو ستون دیگه رو تو خودش نگه می‌داره 'df['TotalAttributes']' # این کار بهمون نشون می‌ده که یه دانشجو چند تا از این دو ویژگی رو داره # df['TotalAttributes'] = df['PassedExam'] + df['EnrolledInCourse']

رو بعد از انجام عملیات نمایش می‌دیم 'DataFrame'، در نهایت # رو ببینید که اضافه شده 'TotalAttributes' می‌تونید ستون جدید # print("دیتاست بعد از انجام عملیات روی ویژگی‌های باینری") print(df)

دیتاست اصلی:

	StudentID	PassedExam	EnrolledInCourse
0	1	1	1
1	2	0	1
2	3	1	0
3	4	1	1
4	5	0	0

PassedExam: تعداد هر مقدار برای
PassedExam
1 3
0 2
Name: count, dtype: int64

EnrolledInCourse: تعداد هر مقدار برای
EnrolledInCourse
1 3
0 2
Name: count, dtype: int64

دیتاست بعد از انجام عملیات روی ویژگی‌های باینری:

	StudentID	PassedExam	EnrolledInCourse	TotalAttributes
0	1	1	1	2
1	2	0	1	1
2	3	1	0	1
3	4	1	1	2
4	5	0	0	0



توضیحات مثال کد پایتون

در این مثال:

- ما یک دیتافریم (DataFrame) پانداس با دو ویژگی دودویی: 'PassedExam' (آزمون گذرانده شده) و 'EnrolledInCourse' (در دوره ثبت‌نام شده) ایجاد می‌کنیم.
- ما تعداد رکوردهای هر مقدار منحصربه‌فرد را در ویژگی‌های دودویی با استفاده از تابع 'value_counts()' کاوش می‌کنیم.
- ما یک عملیات پایه (جمع) را روی ویژگی‌های دودویی انجام می‌دهیم تا یک ویژگی جدید به نام 'TotalAttributes' (مجموع ویژگی‌ها) ایجاد کنیم.
- دیتافریم نهایی (df) پس از این عملیات نمایش داده می‌شود.

2.1.3. Ordinal attributes



ویژگی‌های ترتیبی

ویژگی‌های ترتیبی (Ordinal Attributes) متغیرهای کیفی هستند که دارای یک ترتیب یا رتبه‌بندی معنادار بین دسته‌ها

می‌باشند. برخلاف ویژگی‌های اسمی، ویژگی‌های ترتیبی یک توالی روشن و معنادار دارند، اما لزوماً فواصل بین آن‌ها یکنواخت یا

به‌خوبی تعریف‌شده نیست. نمونه‌هایی از ویژگی‌های ترتیبی شامل **سطوح تحصیلی** (مانند ابتدایی، دبیرستان، دانشگاه)،

رتبه‌بندی رضایت مشتری یا **طبقات اجتماعی-اقتصادی** می‌شوند.

کد پایتون کاربردی برای ویژگی‌های ترتیبی 📊

بیاييد یک قطعه کد پایتون با استفاده از کتابخانه **pandas** برای کار با یک مجموعه داده حاوی ویژگی‌های ترتیبی ایجاد کنیم. ما

یک مجموعه داده نمونه را بارگذاری کرده، ویژگی‌های ترتیبی را کاوش می‌کنیم و نحوه رمزگذاری آن‌ها را برای حفظ رابطه ترتیبی

نمایش می‌دهیم.

In [4]: **import** pandas **as** pd

```
# رو وارد می‌کنه 'pandas' این قسمت اول کده و کتابخونه
# يه ابزار قدرتمند برای کار با داده‌هاست، مخصوصاً وقتی داده‌ها به شکل جدول باشن 'pandas'
# استفاده می‌کنیم 'DataFrame' ما از این کتابخونه برای ساختن و دستکاری

# هستن (Ordinal) این دیکشنری داده‌های نمونه ما رو داره. این بار، داده‌ها از نوع ترتیبی
# ویژگی‌های ترتیبی دسته‌بندی‌هایی هستن که بینشون یه ترتیب یا رتبه مشخص وجود داره
# پایین‌تره 'College' از 'High School' پایین‌تره و 'High School' از 'Elementary' یه ترتیب منطقی داره 'EducationLevel' مثلاً
# هم یک ویژگی ترتیبیه، چون ۵ از ۴ بهتره و ۴ از ۳
data = {
    'StudentID': [1, 2, 3, 4, 5],
    'EducationLevel': ['High School', 'College', 'Elementary', 'College', 'High School'],
    'SatisfactionRating': [3, 5, 2, 4, 1]
}

# جدید می‌سازیم 'DataFrame' با استفاده از داده‌های بالا، یه
# باهاش کار می‌کنه 'pandas' همون جدول داده‌هاست که 'DataFrame'
df = pd.DataFrame(data)

# اصلی رو چاپ می‌کنیم تا ببینیم داده‌های خام چه شکلی هستن 'DataFrame'
print("دیتاست اصلی:")
print(df)
print("\n")

# این یه لیست از ستون‌های ترتیبیه که می‌خوایم بررسی‌شون کنیم
ordinal_attributes = ['EducationLevel', 'SatisfactionRating']

# ساده‌اس که روی ستون‌های ترتیبی می‌چرخه (for loop) این یه حلقه
# مقادیر منحصر به فرد هر ستون رو پیدا می‌کنه و بهمون نشون می‌ده 'unique' تابع
# با دیدن مقادیر، می‌تونیم ترتیب موجود بین اون‌ها رو حدس بزنیم
for attribute in ordinal_attributes:
    print(f"{attribute}: {df[attribute].unique()}")

# رو انجام می‌ده (Encoding) اینجا مهم‌ترین بخش کده که عملیات رمزگذاری
# ساختیم که هر سطح تحصیلی رو به یه عدد مرتبط تبدیل می‌کنه 'education_level_mapping' ما یه دیکشنری
# می‌شن (map) به ۳ نگاشت 'College' به ۲ و 'High School' به عدد ۱ 'Elementary' مثلاً
# این کار داده‌های متنی رو به اعدادی تبدیل می‌کنه که ترتیب رو حفظ می‌کنن
education_level_mapping = {'Elementary': 1, 'High School': 2, 'College': 3}
df['EducationLevelEncoded'] = df['EducationLevel'].map(education_level_mapping)

# بعد از عملیات رمزگذاری هست 'DataFrame' نهایی برای نمایش 'print'
# اضافه شده که مقادیر عددی ترتیب‌دار رو داره 'EducationLevelEncoded' می‌تونید ببینید که یک ستون جدید به اسم
print("\n دیتاست بعد از رمزگذاری ویژگی‌های ترتیبی:")
print(df)
```

دیتاست اصلی:

	StudentID	EducationLevel	SatisfactionRating
0	1	High School	3
1	2	College	5
2	3	Elementary	2
3	4	College	4
4	5	High School	1

EducationLevel: ['High School' 'College' 'Elementary'] مقادیر یکتا در

SatisfactionRating: [3 5 2 4 1] مقادیر یکتا در

دیتاست بعد از رمزگذاری ویژگی‌های ترتیبی:

	StudentID	EducationLevel	SatisfactionRating	EducationLevelEncoded
0	1	High School	3	2
1	2	College	5	3
2	3	Elementary	2	1
3	4	College	4	3
4	5	High School	1	2



توضیحات مثال کد پایتون

در این مثال:

- ما یک دیتافریم (DataFrame) پانداس با دو ویژگی ترتیبی: 'EducationLevel' (سطح تحصیلات) و 'SatisfactionRating' (امتیاز رضایت) ایجاد می‌کنیم.
- ما مقادیر منحصربه‌فرد (Unique Values) را در هر ویژگی ترتیبی کاوش می‌کنیم.
- ما ویژگی‌های ترتیبی را با استفاده از تابع **map()** به مقادیر عددی معنادار رمزگذاری می‌کنیم تا یک ویژگی جدید به نام 'EducationLevelEncoded' ایجاد شود.
- دیتافریم نهایی (**df**) پس از این عملیات نمایش داده می‌شود.

2.1.4. Numeric attributes

ویژگی‌های عددی

ویژگی‌های عددی (Numeric Attributes) نمایانگر کمیت‌ها هستند و می‌توانند مقادیر عددی را بپذیرند. این ویژگی‌ها به دو دسته اصلی تقسیم می‌شوند: گسسته (Discrete) و پیوسته (Continuous).

ویژگی‌های عددی گسسته تنها می‌توانند مقادیر متمایز و جداگانه را بپذیرند (مثلاً تعداد اتاق خواب در یک خانه 🏠)، در حالی که **ویژگی‌های عددی پیوسته** می‌توانند هر مقداری را در یک بازه معین داشته باشند (مثلاً قد 📏 یا وزن).

کد پایتون کاربردی برای ویژگی‌های عددی

بیا یک قطعه کد پایتون با استفاده از کتابخانه **pandas** برای کار با یک مجموعه داده حاوی ویژگی‌های عددی ایجاد کنیم. ما یک مجموعه داده نمونه را بارگذاری کرده، ویژگی‌های عددی را کاوش می‌کنیم و عملیات‌های پایه را روی آن‌ها انجام می‌دهیم.

In [5]:

```
import pandas as pd

# رو وارد می‌کنه 'pandas' این قسمت اول کده و کتابخونه
# یه ابزار خیلی قدرتمنده برای کار کردن با داده‌ها و جدول‌ها 'pandas'

# این دیکشنری حاوی داده‌های نمونه ماست.
# هستن. یعنی مقادیرشون اعداد واقعی هستن که می‌تونیم روشون عملیات ریاضی انجام بدیم (Numeric) این بار، داده‌ها از نوع عددی
# همگی ویژگی‌های عددی هستن (امتیاز) 'Score' و (قد) 'Height (cm)', (سن) 'Age' مثلاً
data = {
    'StudentID': [1, 2, 3, 4, 5],
    'Age': [21, 19, 22, 20, 23],
    'Height (cm)': [175, 160, 180, 165, 185],
    'Score': [85, 92, 78, 89, 95]
}

# جدید می‌سازیم 'DataFrame' با استفاده از داده‌های بالا، به
# باهش کار می‌کنه 'pandas' همون جدولیه که 'DataFrame'
df = pd.DataFrame(data)

# اصلی رو چاپ می‌کنیم تا ببینیم داده‌های خام چه شکلی هستن 'DataFrame'
print("دیتاست اصلی")
print(df)
print("\n")

# این یه لیست از ستون‌های عددیه که می‌خوایم روی اون‌ها عملیات انجام بدیم
numeric_attributes = ['Age', 'Height (cm)', 'Score']

# یه تابع خیلی مفیده که خلاصه‌ای از آمار توصیفی رو برای ستون‌های عددی بهمون می‌ده 'describe()'
# و چارک‌ها رو محاسبه می‌کنه (max) حداکثر، (min) حداقل، (std) انحراف معیار، (mean) مثلاً میانگین
# این آمار بهمون کمک می‌کنه که به دید کلی از توزیع داده‌ها به دست بیاریم
print("آمار توصیفی برای ویژگی‌های عددی:")
print(df[numeric_attributes].describe())

# نرمال‌سازی یه تکنیک پیش‌پردازشه که مقادیر عددی رو طوری تغییر می‌ده که میانگینشون صفر و انحراف معیارشون یک بشه
# این کار برای بعضی از الگوریتم‌های یادگیری ماشین که به مقیاس داده‌ها حساس هستن، ضروریه
# فرمولش اینه: مقدار نرمال‌شده = (مقدار اصلی - میانگین) / انحراف معیار
df['NormalizedScore'] = (df['Score'] - df['Score'].mean()) / df['Score'].std()

# رو بعد از انجام عملیات نمایش می‌دیم 'DataFrame'، در نهایت
# رو ببینید که مقادیرش نرمال‌سازی شدن 'NormalizedScore' می‌تونید ستون جدید
print("\ndیتاست بعد از انجام عملیات روی ویژگی‌های عددی:")
print(df)
```

دیتاست اصلی:

	StudentID	Age	Height (cm)	Score
0	1	21	175	85
1	2	19	160	92
2	3	22	180	78
3	4	20	165	89
4	5	23	185	95

آمار توصیفی برای ویژگی‌های عددی:

	Age	Height (cm)	Score
count	5.000000	5.000000	5.000000
mean	21.000000	173.000000	87.800000
std	1.581139	10.368221	6.610598
min	19.000000	160.000000	78.000000
25%	20.000000	165.000000	85.000000
50%	21.000000	175.000000	89.000000
75%	22.000000	180.000000	92.000000
max	23.000000	185.000000	95.000000

دیتاست بعد از انجام عملیات روی ویژگی‌های عددی:

	StudentID	Age	Height (cm)	Score	NormalizedScore
0	1	21	175	85	-0.423562
1	2	19	160	92	0.635343
2	3	22	180	78	-1.482468
3	4	20	165	89	0.181527
4	5	23	185	95	1.089160



توضیحات مثال کد پایتون

در این مثال:

- ما یک دیتافریم (DataFrame) پانداس با سه ویژگی عددی: 'Age' (سن)، 'Height (cm)' (قد به سانتی‌متر) و 'Score' (امتیاز) ایجاد می‌کنیم.
- ما با استفاده از تابع **describe()**، آمارهای خلاصه (Summary Statistics) ویژگی‌های عددی را کاوش می‌کنیم.
- ما یک عملیات پایه (نرمال‌سازی) را روی ویژگی 'Score' انجام می‌دهیم تا یک ویژگی جدید به نام 'NormalizedScore' (امتیاز نرمال‌شده) ایجاد کنیم.
- دیتافریم نهایی (**df**) پس از این عملیات نمایش داده می‌شود.

2.1.5. Discrete vs. continuous attributes

1234

ویژگی‌های گسسته و پیوسته

ویژگی‌های گسسته (Discrete Attributes):

تعریف: ویژگی‌های گسسته تنها می‌توانند مقادیر مشخص و جداگانه را بپذیرند.

مثال‌ها: تعداد اتاق خواب در یک خانه 🏠، تعداد آیتم‌ها در یک سبد خرید 🛒، تعداد دانشجو در یک کلاس 👨🎓.

ماهیت: این ویژگی‌ها اغلب با اعداد کامل (صحیح) شمارش می‌شوند و دارای مرزهای مشخصی بین مقادیر هستند.

ویژگی‌های پیوسته (Continuous Attributes):

تعریف: ویژگی‌های پیوسته می‌توانند هر مقداری را در یک بازه معین داشته باشند.

مثال‌ها: قد 📏، وزن ⚖️، دما 🌡️ و هر نوع اندازه‌گیری که می‌تواند مقادیر اعشاری داشته باشد.

ماهیت: این ویژگی‌ها دارای مجموعه‌ای پیوسته و نامحدود از مقادیر ممکن هستند، که آن‌ها را برای اندازه‌گیری مناسب می‌سازد.

1234

کد پایتون کاربردی برای ویژگی‌های گسسته و پیوسته

بیا یید یک قطعه کد پایتون با استفاده از کتابخانه **pandas** برای کار با یک مجموعه داده حاوی هر دو ویژگی گسسته و پیوسته ایجاد کنیم. ما یک مجموعه داده نمونه را بارگذاری کرده، ماهیت هر نوع را کاوش می‌کنیم و عملیات‌های پایه را روی آن‌ها انجام می‌دهیم.

In [6]: **import** pandas **as** pd

```
# رو وارد می‌کنه 'pandas' این قسمت اول کده و کتابخونه
# به ابزار خیلی پرکاربرده برای کار کردن با داده‌ها و جدول‌ها 'pandas'

# حاوی داده‌های نمونه ماست (dictionary) این دیکشنری
# هستن (Continuous) و هم از نوع پیوسته (Discrete) این بار، داده‌ها هم از نوع گسسته
# (تعداد درس‌ها) 'NumCourses' مقادیری رو می‌گیرن که قابل شمارش هستن، مثل (Discrete) ویژگی‌های گسسته
# (درآمد) 'Income' یا (معدل) 'GPA' مقادیری رو می‌گیرن که می‌تونن هر عددی باشن، مثل (Continuous) ویژگی‌های پیوسته
data = {
    'StudentID': [1, 2, 3, 4, 5],
    'NumCourses': [4, 5, 3, 6, 4], # (تعداد درس‌ها)
    'GPA': [3.5, 4.0, 3.2, 3.8, 3.9], # (معدل)
    'Income': [25000, 30000, 20000, 35000, 32000] # (درآمد به دلار)
}

# جدید می‌سازیم 'DataFrame' با استفاده از داده‌های بالا، به
# 'DataFrame' همون جدولیه که
df = pd.DataFrame(data)

# اصلی رو چاپ می‌کنیم تا ببینیم داده‌های خام چه شکلی هستن 'DataFrame'
print("دیتاست اصلی:")
print(df)
print("\n")

# این بخش نوع ویژگی‌ها رو بررسی و چاپ می‌کنه
print("نوع ویژگی‌ها")
# می‌چرخیم 'DataFrame' روی ستون‌های 'for'، با به حلقه
# می‌تونیم نوع داده هر ستون رو بفهمیم 'df[column].dtype' با
# باشه، می‌گیم گسسته‌اس (عدد صحیح) 'int' اگه نوع داده
# باشه، می‌گیم پیوسته‌اس (عدد اعشاری) 'float' اگه نوع داده
for column in df.columns:
    if df[column].dtype == 'int':
        print(f"{column} یک ویژگی گسسته است")
    elif df[column].dtype == 'float':
        print(f"{column} یک ویژگی پیوسته است")

# انجام می‌دیم 'Income' اینجا یک عملیات ساده روی ویژگی پیوسته
# رو بر ۱۰۰۰ تقسیم می‌کنه 'Income' می‌سازیم که مقدار 'ScaledIncome' ستون جدیدی به اسم
# (Scaling) 'این کار داده‌ها رو برای خوندن راحت‌تر می‌کنه و بهش می‌گن 'مقیاس‌گذاری
df['ScaledIncome'] = df['Income'] / 1000 # مقیاس‌گذاری درآمد برای خوانایی بیشتر

# رو بعد از انجام عملیات نمایش می‌دیم 'DataFrame'، در نهایت
# رو ببینید که اضافه شده 'ScaledIncome' می‌تونید ستون جدید
print("\ndیتاست بعد از انجام عملیات روی ویژگی‌های پیوسته")
print(df)
```

دیتاست اصلی:

	StudentID	NumCourses	GPA	Income
0	1	4	3.5	25000
1	2	5	4.0	30000
2	3	3	3.2	20000
3	4	6	3.8	35000
4	5	4	3.9	32000

نوع ویژگی‌ها:

یک ویژگی پیوسته است GPA

دیتاست بعد از انجام عملیات روی ویژگی‌های پیوسته:

	StudentID	NumCourses	GPA	Income	ScaledIncome
0	1	4	3.5	25000	25.0
1	2	5	4.0	30000	30.0
2	3	3	3.2	20000	20.0
3	4	6	3.8	35000	35.0
4	5	4	3.9	32000	32.0



توضیحات مثال کد پایتون

در این مثال:

- ما یک دیتافریم (DataFrame) پانداس با سه ویژگی: 'NumCourses' (تعداد دوره‌ها) از نوع گسسته، 'GPA' از نوع پیوسته، و 'Income' (درآمد) از نوع پیوسته ایجاد می‌کنیم.
- ما ماهیت هر ویژگی را بر اساس نوع داده آن کاوش می‌کنیم.
- ما یک عملیات پایه (Scaling) را روی یک ویژگی پیوسته ('Income') انجام می‌دهیم تا یک ویژگی جدید به نام 'ScaledIncome' (درآمد مقیاس‌بندی‌شده) ایجاد شود.
- دیتافریم نهایی (df) پس از این عملیات نمایش داده می‌شود.

Section 2.2 (Statistics of data)

2.2.1. Measuring the central tendency



معیارهای گرایش به مرکز

اندازه‌گیری گرایش به مرکز، روشی است برای خلاصه کردن یک مجموعه داده با شناسایی مقدار مرکزی یا میانگین آن. سه معیار متداول برای گرایش به مرکز وجود دارد: میانگین (mean)، میانه (median) و نما (mode).



توضیح معیارهای گرایش به مرکز

میانگین (Average):

این معیار با جمع کردن تمام مقادیر یک مجموعه داده و تقسیم حاصل جمع بر تعداد مقادیر محاسبه می‌شود.

میانه (Median):

میانه مقدار میانی یک مجموعه داده است، زمانی که به ترتیب صعودی یا نزولی مرتب شده باشد. اگر مجموعه داده دارای تعداد زوجی از مقادیر باشد، میانه، میانگین دو مقدار میانی است.

نما (Mode):

نما مقداری است که بیشترین تکرار را در یک مجموعه داده دارد.



کد پایتون کاربردی برای معیارهای گرایش به مرکز

بیا یک قطعه کد پایتون با استفاده از کتابخانه **pandas** برای محاسبه معیارهای گرایش به مرکز (میانگین، میانه و نما) در یک مجموعه داده نمونه ایجاد کنیم. ما یک دیتافریم را بارگذاری کرده، این معیارها را محاسبه و نتایج را نمایش می‌دهیم.

In [7]: `import numpy as np`

```
# رو وارد می‌کنه 'numpy' این قسمت اول کده و کتابخونه
# یه کتابخونه خیلی مهم برای کار با آرایه‌ها و عملیات ریاضی روی آون‌هاست 'numpy'
# بیشتر عملیات آماری و علمی تو پایتون با این کتابخونه انجام می‌شه.
```

```
# از داده‌های نمونه ماست (array) این به آرایه
# آرایه یه ساختار داده‌ایه که مقادیر عددی رو تو خودش نگه می‌داره
data = np.array([15, 18, 2, 36, 12, 25, 18, 40, 28, 22])
```

```
# داده‌ها رو حساب می‌کنه (Mean) این بخش، میانگین
# میانگین همون معدل یا متوسط حسابی داده‌هاست
# هست که میانگین رو برامون محاسبه می‌کنه 'numpy' تابعی از 'np.mean()'
mean_value = np.mean(data)
print(f"میانگین: {mean_value}")
```

```
# داده‌ها رو محاسبه می‌کنیم (Median) اینجا میانه
# میانه، مقدار وسطی داده‌هاست، بعد از اینکه آون‌ها رو مرتب کنیم
# قرار می‌گیره 'Outliers' این معیار، کمتر از میانگین تحت تأثیر داده‌های پرت یا
# تابعیه که میانه رو حساب می‌کنه 'np.median()'
median_value = np.median(data)
print(f"میانه: {median_value}")
```

```
# یا پرتکرارترین مقدار رو پیدا می‌کنه (Mode) این قسمت مد
# هم مقدار با بیشترین تکرار رو پیدا می‌کنه 'np.argmax()' تعداد هر مقدار رو می‌شماره و 'np.bincount()'
# در واقع، مد، مقداریه که تو مجموعه داده‌ها از همه بیشتر تکرار شده
mode_value = np.argmax(np.bincount(data))
print(f"مد: {mode_value}")
```

توضیحات مثال کد پایتون

- اگر هنوز کتابخانه NumPy را نصب نکرده‌اید، مطمئن شوید که با اجرای دستور `pip install numpy` در محیط پایتون خود این کار را انجام داده‌اید.
- این کد از توابع NumPy برای محاسبه میانگین، میانه و نمای مجموعه داده استفاده می‌کند. شما می‌توانید آرایه داده را با مجموعه داده خود جایگزین کنید.

2.2.2. Measuring the dispersion of data

اندازه‌گیری پراکندگی داده

اندازه‌گیری پراکندگی داده برای درک میزان پخش‌شدگی یا خوشه‌بندی مقادیر در یک مجموعه داده مهم است. چندین معیار پراکندگی وجود دارد، از جمله دامنه (range)، واریانس (variance) و انحراف معیار (standard deviation).

اندازه‌گیری پراکندگی داده

اندازه‌گیری پراکندگی داده برای درک میزان پخش‌شدگی یا خوشه‌بندی مقادیر در یک مجموعه داده مهم است. چندین معیار پراکندگی وجود دارد، از جمله دامنه (range)، واریانس (variance) و انحراف معیار (standard deviation).

دامنه (Range):

دامنه، اختلاف بین حداکثر و حداقل مقادیر در یک مجموعه داده است. این معیار یک اندازه‌گیری ساده از میزان پخش‌شدگی مقادیر ارائه می‌دهد.

واریانس (Variance):

واریانس، میانگین اختلاف مجذور مقادیر از میانگین مجموعه داده است. واریانس بالاتر نشان‌دهنده پراکندگی بیشتر است.

انحراف معیار (Standard Deviation):

انحراف معیار، ریشه دوم واریانس است. این معیار یک سنجش قابل تفسیرتر از پراکندگی داده‌ها ارائه می‌دهد، زیرا واحد آن با واحد داده اصلی یکسان است.

کد پایتون کاربردی برای اندازه‌گیری پراکندگی داده

بیا یک قطعه کد پایتون با استفاده از کتابخانه **pandas** برای محاسبه معیارهای پراکندگی داده (دامنه، واریانس و انحراف معیار) در یک مجموعه داده نمونه ایجاد کنیم. ما یک دیتافریم را بارگذاری کرده، این معیارها را محاسبه و نتایج را نمایش می‌دهیم.

In [8]: `import numpy as np`

رو وارد می‌کنه 'numpy' این قسمت اول کده و کتابخونه #
یه کتابخونه خیلی مهم برای کار با آرایه‌ها و عملیات ریاضی روی اون‌هاست 'numpy' #

.بیشتر عملیات آماری و علمی تو پایتون با این کتابخونه انجام می‌شه #

```
# از داده‌های نمونه ماست (array) این یه آرایه #
# آرایه یه ساختار داده‌ایه که مقادیر عددی رو تو خودش نگه می‌داره #
data = np.array([15, 18, 2, 36, 12, 25, 18, 40, 28, 22])
```

```
# داده‌ها رو حساب می‌کنه (Mean) این بخش، میانگین #
# میانگین همون معدل یا متوسط حسابی داده‌هاست #
# هست که میانگین رو برامون محاسبه می‌کنه 'numpy' تابعی از 'np.mean()'
mean_value = np.mean(data)
print(f"میانگین: {mean_value}")
```

```
# داده‌ها رو محاسبه می‌کنیم (Median) اینجا میانه #
# میانه، مقدار وسطی داده‌هاست، بعد از اینکه اون‌ها رو مرتب کنیم #
# قرار می‌گیره 'Outliers' این معیار، کمتر از میانگین تحت تأثیر داده‌های پرت یا #
# تابعیه که میانه رو حساب می‌کنه 'np.median()'
median_value = np.median(data)
print(f"میانه: {median_value}")
```

```
# با پرتکارترین مقدار رو پیدا می‌کنه (Mode) این قسمت مد #
# هم مقدار با بیشترین تکرار رو پیدا می‌کنه 'np.argmax()' تعداد هر مقدار رو می‌شماره و 'np.bincount()'
# در واقع، مد، مقداریه که تو مجموعه داده‌ها از همه بیشتر تکرار شده #
mode_value = np.argmax(np.bincount(data))
print(f"مد: {mode_value}")
```

```
# داده‌ها رو حساب می‌کنه (Range) این بخش، دامنه #
# دامنه، تفاوت بین بزرگترین و کوچکترین مقدار تو مجموعه داده‌هاست #
# تابعیه که این مقدار رو حساب می‌کنه 'np.ptp()'
# دامنه بهمون یه دید کلی از پراکندگی داده‌ها می‌ده #
data_range = np.ptp(data)
print(f"دامنه: {data_range}")
```

```
# داده‌ها رو محاسبه می‌کنیم (Variance) اینجا واریانس #
# واریانس، میانگین مربع اختلاف هر داده با میانگین کل داده‌هاست #
# این معیار نشون می‌ده که داده‌ها چقدر حول میانگین پراکنده شدن #
# تابعیه که واریانس رو حساب می‌کنه 'np.var()'
data_variance = np.var(data)
print(f"واریانس: {data_variance}")
```

```
# داده‌ها رو پیدا می‌کنه (Standard Deviation) این قسمت انحراف معیار #
# انحراف معیار، ریشه دوم واریانس و همون واحدهای اصلی داده‌ها رو داره #
# این معیار هم میزان پراکندگی داده‌ها رو نشون می‌ده. هرچی انحراف معیار بیشتر باشه، داده‌ها پراکنده‌ترن #
# تابعیه که انحراف معیار رو محاسبه می‌کنه 'np.std()'
data_stddev = np.std(data)
print(f"انحراف معیار: {data_stddev}")
```

میانگین: 21.6
میانه: 20.0
مد: 18
دامنه: 38
واریانس: 114.44000000000001
انحراف معیار: 10.697663296253065



توضیحات مثال کد پایتون

در این کد، `np.ptp` دامنه (range)، `np.var` واریانس (variance) و `np.std` انحراف معیار (standard deviation) را محاسبه می‌کند. آرایه داده را با مجموعه داده خود جایگزین کنید.

2.2.3. Covariance and correlation analysis



کوواریانس و تحلیل همبستگی

کوواریانس (Covariance) و **تحلیل همبستگی (Correlation Analysis)** معیارهای آماری هستند که میزان تغییر همزمان دو متغیر را توصیف می‌کنند.



کوواریانس و همبستگی

کوواریانس (Covariance):

کوواریانس میزان تغییر مقادیر دو متغیر را نسبت به یکدیگر اندازه‌گیری می‌کند. یک کوواریانس مثبت نشان می‌دهد که با افزایش یک متغیر، متغیر دیگر نیز تمایل به افزایش دارد و برعکس برای کوواریانس منفی. با این حال، مقیاس کوواریانس استاندارد نیست، که تفسیر قدرت رابطه را دشوار می‌سازد.

همبستگی (Correlation):

همبستگی یک معیار استاندارد برای سنجش قدرت و جهت رابطه خطی بین دو متغیر است. ضریب همبستگی در بازه -1 تا 1 قرار دارد. ضریب همبستگی 1 نشان‌دهنده یک همبستگی مثبت کامل، -1 نشان‌دهنده یک همبستگی منفی کامل، و 0 نشان‌دهنده عدم وجود همبستگی خطی است.

کد پایتون کاربردی برای تحلیل همبستگی و کوواریانس

بیا یک قطعه کد پایتون با استفاده از کتابخانه **pandas** برای محاسبه معیارهای **کوواریانس** و **همبستگی** در یک مجموعه داده نمونه ایجاد کنیم. ما یک دیتافریم را بارگذاری کرده، این معیارها را محاسبه و نتایج را نمایش می‌دهیم.

```
In [9]: import numpy as np
import pandas as pd

# رو وارد می‌کنه 'pandas' و 'numpy' این قسمت اول کده و کتابخونه‌های
# برای کار با داده‌ها تو قالب جدول استفاده می‌شه 'pandas' برای کار با آرایه‌ها و محاسبات ریاضی پیشرفته و 'numpy'.

# این به دیکشنری با داده‌های نمونه ماست
# هستن که می‌خوایم رابطه‌شون رو با هم بررسی کنیم 'Variable2' و 'Variable1' این داده‌ها شامل دو متغیر
data = {
    'Variable1': [15, 18, 2, 36, 12, 25, 18, 40, 28, 22],
    'Variable2': [10, 12, 5, 30, 8, 20, 15, 35, 25, 18]
}

# تبدیل می‌کنیم 'DataFrame' داده‌ها رو به یه 'pandas' با استفاده از
# همون جدولیه که داده‌ها توش منظم و مرتب هستن 'DataFrame'.
df = pd.DataFrame(data)

# رو محاسبه می‌کنه '**(Covariance Matrix)' این بخش، '**ماتریس کوواریانس
# نس، میزان تغییر هم‌جهت دو متغیر رو نشون می‌ده. اگه دو متغیر با هم زیاد بشن یا کم بشن، کوواریانسشون مثبت می‌شه و اگه یکی زیاد و دیگری کم بشه، کوواریانسشون منفی می‌شه
# هست که ماتریس کوواریانس رو حساب می‌کنه 'numpy' تابعی از 'np.cov()'
# هم به تابع می‌گه که هر ستون یک متغیره، نه هر سطر 'rowvar=False'
cov_matrix = np.cov(df, rowvar=False)
print(f"ماتریس کوواریانس:\n{cov_matrix}")

# رو محاسبه می‌کنه '**(Correlation Matrix)' این قسمت '**ماتریس همبستگی
# همبستگی هم مثل کوواریانس، با این تفاوت که مقادیرش بین -1 و 1 هستن
# این مقدار نشون می‌ده که رابطه بین دو متغیر چقدر قوی و در چه جهتی
# اگه مقدار نزدیک به 1 باشه، همبستگی مثبت و قویه. اگه نزدیک به -1 باشه، همبستگی منفی و قویه. اگه نزدیک به 0 باشه، همبستگی ضعیفه
# هست که ماتریس همبستگی رو حساب می‌کنه 'pandas' تابعی از 'df.corr()'
correlation_matrix = df.corr()
print(f"ماتریس همبستگی:\n{correlation_matrix}")

# ماتریس کوواریانس:
[[127.15555556 108.13333333]
 [108.13333333 95.95555556]]
# ماتریس همبستگی:
      Variable1  Variable2
Variable1    1.000000    0.978942
Variable2    0.978942    1.000000
```

توضیحات مثال کد پایتون

در این کد، **np.cov** ماتریس کوواریانس و متد **corr()** یک دیتافریم پانداس ماتریس همبستگی را محاسبه می‌کند. ستون‌های **Variable1** و **Variable2** را با متغیرهای دلخواه خود جایگزین کنید.

2.2.4. Graphic displays of basic statistics of data

نمایش‌های گرافیکی آمار پایه

نمایش‌های گرافیکی (**Graphic Displays**) از آمارهای پایه، برای بصری‌سازی ویژگی‌های یک مجموعه داده ضروری هستند. این نمایش‌ها می‌توانند به درک بهتر توزیع، گرایش به مرکز و پراکندگی داده‌ها کمک کنند.

هیستوگرام‌ها (Histograms):

توضیح: هیستوگرام‌ها با تقسیم یک مجموعه داده به سطل‌ها (bins) و نمایش فراوانی (frequency) یا احتمال قرار گرفتن مقادیر در هر سطل، یک نمایش بصری از توزیع داده‌ها ارائه می‌دهند.

نمودارهای جعبه‌ای (Box Plots):

توضیح: نمودارهای جعبه‌ای (یا نمودارهای جعبه و خط) یک خلاصه گرافیکی از توزیع یک مجموعه داده، شامل میانه (median)، چنک‌ها (quartiles) و نقاط پرت (outliers) بالقوه را فراهم می‌کنند.

نمودارهای پراکندگی (Scatter Plots):

توضیح: نمودارهای پراکندگی، نقاط داده منفرد را در یک فضای دوبعدی نمایش می‌دهند و شناسایی الگوها، روابط و نقاط پرت بین دو متغیر را آسان می‌سازند.

کد پایتون کاربردی برای نمایش‌های گرافیکی آماری

بیاپید یک قطعه کد پایتون با استفاده از کتابخانه‌های **numpy** و **matplotlib** برای ایجاد نمایش‌های گرافیکی آمار پایه، شامل **هیستوگرام**، **نمودار جعبه‌ای** و **نمودار پراکندگی**، در یک مجموعه داده نمونه ایجاد کنیم. ما یک دیتافریم را بارگذاری کرده، این نمودارها را رسم و نمایش می‌دهیم.

In [11]: **import** matplotlib.pyplot **as** plt
import numpy **as** np

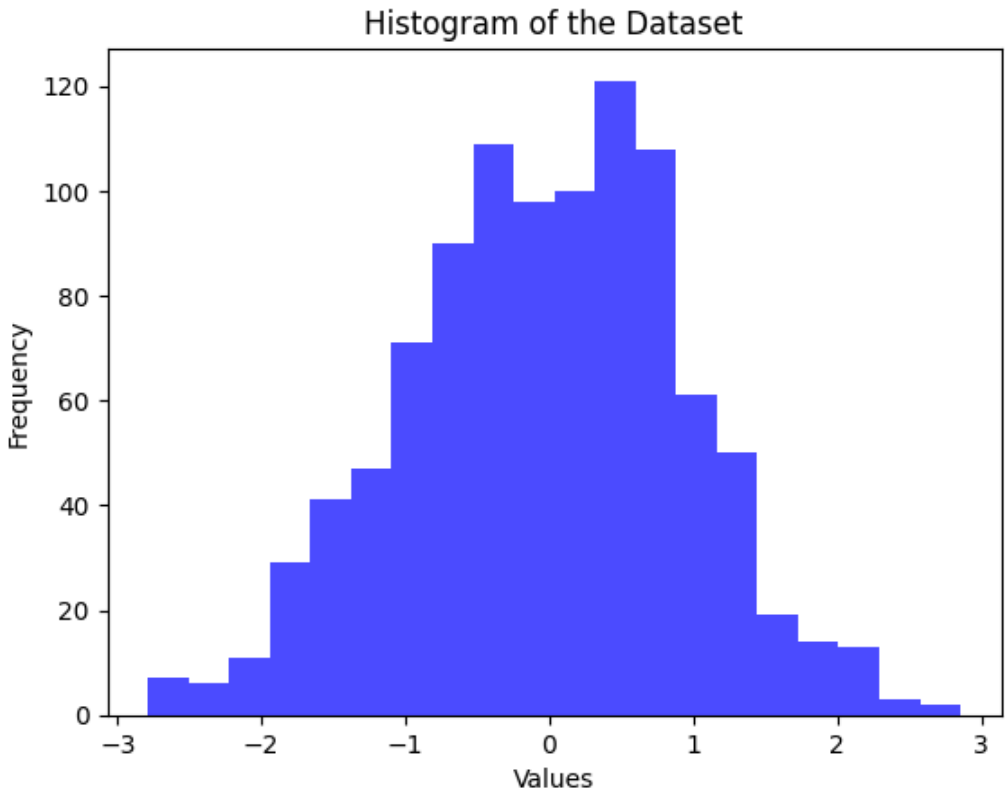
این قسمت اول کده. ما دو تا کتابخونه رو وارد می‌کنیم
'matplotlib.pyplot' که ابزار اصلی ما برای کشیدن نمودار و تصویرسازی داده‌هاست
'numpy' که برای کار با آرایه‌ها و داده‌های عددی استفاده می‌شه

از ۱۰۰۰ عدد تصادفی رو می‌سازه (array) این خط به آرایه
هستن (Normal Distribution) اعدادی رو تولید می‌کنه که بر اساس توزیع نرمال 'np.random.randn()' تابع
یعنی بیشتر اعداد نزدیک به صفر هستن و هرچه از صفر دورتر می‌شیم، تعدادشون کمتر می‌شه
می‌تونن این خط رو با داده‌های خودت جایگزین کنی تا هیستوگرام اون‌ها رو ببینی
data = np.random.randn(1000) # داده‌های خودت رو اینجا جایگزین کن

رو رسم می‌کنه (Histogram) این بخش نمودار هیستوگرام
تابع اصلی برای رسم هیستوگرامه 'plt.hist()'
دسته‌بندی کنه (bin) به نمودار می‌گه که داده‌ها رو توی ۲۰ تا سطل یا ستون 'bins=20'
میزان شفافیتشون رو مشخص می‌کنه 'alpha' رنگ ستون‌ها رو آبی و 'color'
plt.hist(data, bins=20, color='blue', alpha=0.7)

این خطوط برای زیباتر شدن و قابل فهم‌تر شدن نمودار هستن
plt.title('Histogram of the Dataset')
رو می‌ذاره که نشون‌دهنده مقادیر داده‌هاست (X-axis) این خط برچسب محور افقی
plt.xlabel('Values')
یا تعداد تکرار هر مقدار هست (Frequency) رو می‌ذاره که نشون‌دهنده فراوانی (Y-axis) این خط برچسب محور عمودی
plt.ylabel('Frequency')

نمودار رسم‌شده رو به ما نمایش می‌ده 'plt.show()' در نهایت
plt.show()



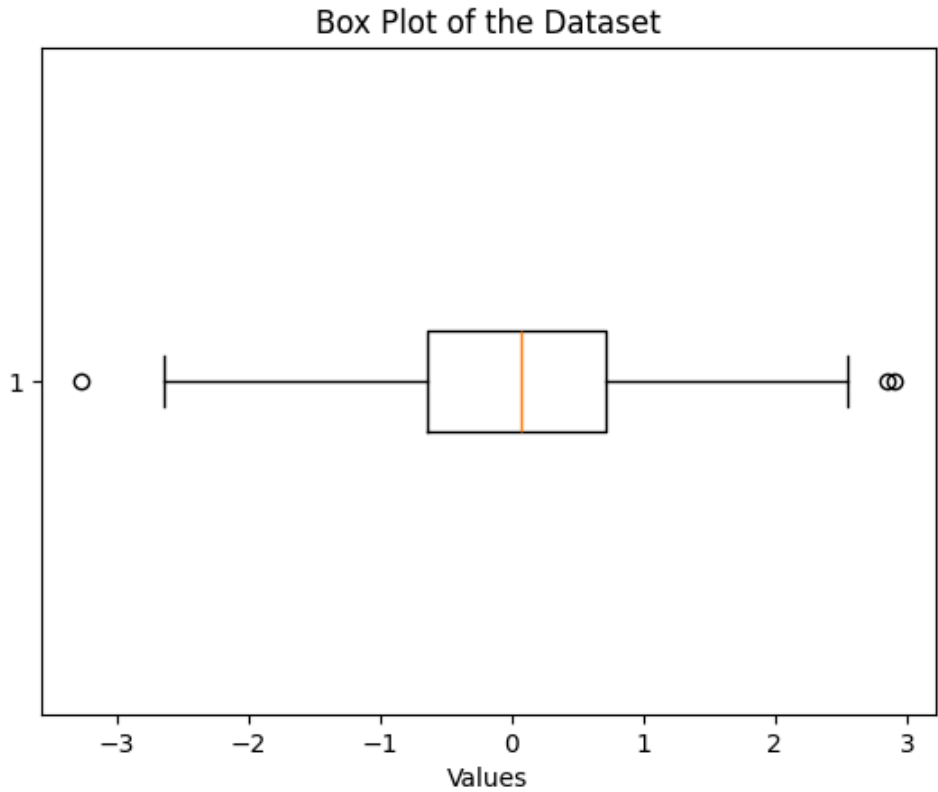
```
In [12]: import matplotlib.pyplot as plt
import numpy as np

# این قسمت اول کده. دو تا کتابخونه رو وارد می‌کنیم
# 'matplotlib.pyplot' داده‌هاست
# 'numpy' داده‌های عددی استفاده می‌شه

# از ۱۰۰۰ عدد تصادفی رو می‌سازه (array) این خط به آرایه
# هستن (Normal Distribution) اعدادی رو تولید می‌کنه که بر اساس توزیع نرمال 'np.random.randn()' تابع
# می‌تونی این خط رو با داده‌های خودت جایگزین کنی تا نمودار جعبه‌ای اون‌ها رو ببینی
data = np.random.randn(1000) # داده‌های خودت رو اینجا جایگزین کن

# رو رسم می‌کنه (Box Plot) این بخش نمودار جعبه‌ای
# نمودار جعبه‌ای به راه خیلی خوب برای دیدن توزیع داده‌هاست
# 'Outliers' مخصوصاً برای پیدا کردن داده‌های پرت یا
# 'plt.boxplot()' تابع اصلی برای رسم این نموداره
# 'vert=False' باعث می‌شه که نمودار به صورت افقی رسم بشه. اگه این رو حذف کنی یا
plt.boxplot(data, vert=False)

# این خطوط برای زیباتر شدن و قابل فهم‌تر شدن نمودار هستن
plt.title('Box Plot of the Dataset')
# 'plt.xlabel()' برچسب محور افقی (X-axis)
plt.xlabel('Values')
```



```
In [13]: import matplotlib.pyplot as plt
import numpy as np

# این قسمت اول کده. ما دو تا کتابخونه رو وارد می‌کنیم
# 'matplotlib.pyplot' داده‌هاست
# 'numpy' داده‌های عددی استفاده می‌شه

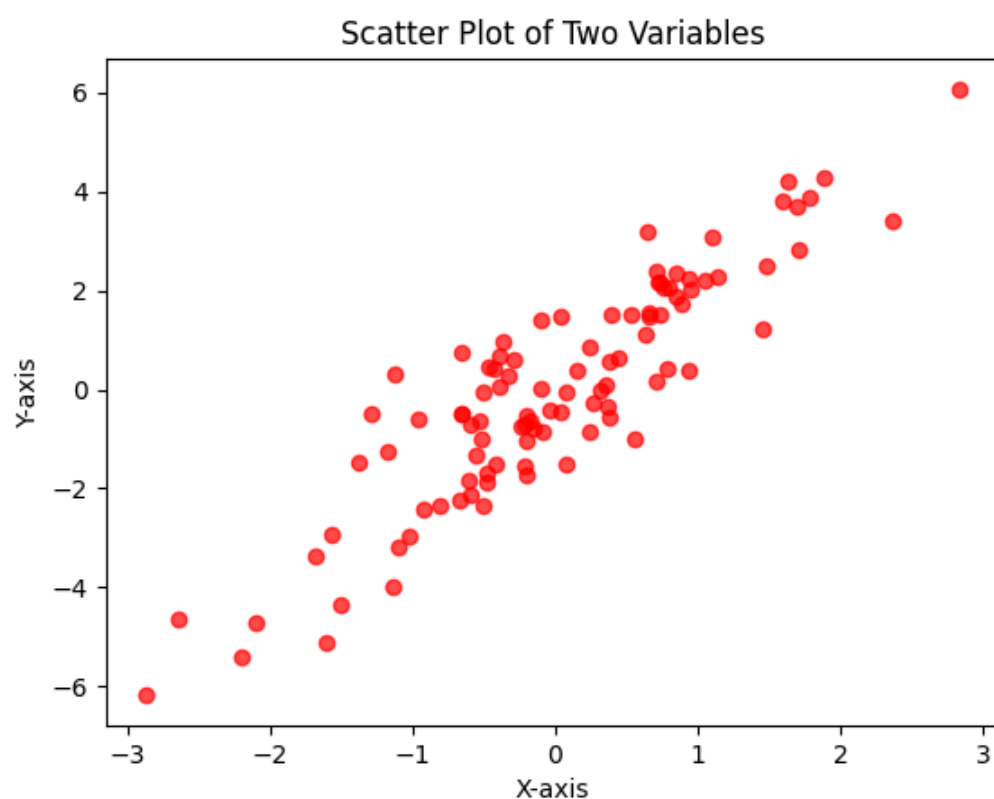
# از داده‌های نمونه رو می‌سازن (array) این خطوط دو تا آرایه
# به آرایه از ۱۰۰ عدد تصادفی که بر اساس توزیع نرمال تولید شده 'x'
# بهش اضافه شده (noise) وابسته است و به مقدار نویز 'x' هم به آرایه از ۱۰۰ عده که به 'y'
# داشته باشیم 'y' و 'x' هدف اینه که به رابطه خطی بین
# می‌تونی این خطوط رو با داده‌های خودت جایگزین کنی تا نمودار پراکندگی اون‌ها رو ببینی
x = np.random.randn(100)
y = 2 * x + np.random.randn(100) # داده‌های خودت رو اینجا جایگزین کن

# رو رسم می‌کنه (Scatter Plot) این بخش نمودار پراکندگی
# نمودار پراکندگی به نمودار دو بعدی برای نمایش رابطه بین دو متغیر عددی
# مربوط به یک نمونه رو نشون می‌ده 'y' و 'x' هر نقطه روی نمودار، مقادیر
```

```
# با این نمودار می‌شه به راحتی رابطه بین دو متغیر رو از نظر بصری دید
# 'plt.scatter()' تابع اصلی برای رسم این نموداره
# 'color' میزان شفافیتشون رو مشخص می‌کنه 'alpha' رنگ نقاط رو قرمز و
plt.scatter(x, y, color='red', alpha=0.7)

# این خطوط برای زیباتر شدن و قابل فهم‌تر شدن نمودار هستن
plt.title('Scatter Plot of Two Variables')
# 'plt.xlabel()' برچسب محور افقی رو می‌ذاره
plt.xlabel('X-axis')
# 'plt.ylabel()' برچسب محور عمودی رو می‌ذاره
plt.ylabel('Y-axis')

# نمودار رسم‌شده رو به ما نمایش می‌ده 'plt.show()'، در نهایت
plt.show()
```



توضیحات مثال کد پایتون

هیستوگرام‌ها (Histograms)، نمودارهای جعبه‌ای (box plots) و نمودارهای پراکندگی (scatter plots) می‌توانند به درک بهتر توزیع داده‌ها، روابط و نقاط پرت کمک کنند.

Section 2.3

2.3.1. Data matrix vs. dissimilarity matrix



ماتریس داده در مقابل ماتریس عدم تشابه

در داده‌کاوی، درک مفاهیم **ماتریس داده (Data Matrix)** و **ماتریس عدم تشابه (Dissimilarity Matrix)** بسیار حیاتی است.

ماتریس داده:

توضیح: ماتریس داده یک نمایش ساختاریافته از یک مجموعه داده است که در آن سطرها مربوط به مشاهدات یا نمونه‌های منفرد و ستون‌ها نمایانگر ویژگی‌ها یا صفات مختلف هستند. هر خانه در این ماتریس حاوی مقدار یک ویژگی خاص برای یک نمونه مشخص است.

ماتریس عدم تشابه:

توضیح: ماتریس عدم تشابه، عدم تشابه یا تشابه بین جفت‌های نمونه در یک مجموعه داده را نشان می‌دهد. این ماتریس میزان تفاوت یا شباهت دو نمونه را بر اساس یک معیار عدم تشابه، مانند فاصله یا امتیاز عدم تشابه، کمی‌سازی می‌کند.

```
In [14]: import pandas as pd

# رو وارد می‌کنه 'pandas' این قسمت اول کده و کتابخونه
# 'pandas' ابزار اصلی ما برای کار با داده‌هاست، به خصوص وقتی داده‌ها تو قالب جدول باشن

# حاوی داده‌های نمونه ماست (dictionary) این به دیکشنری
# رو نشون می‌ده، مثل یک دانشجو (Object) یا شیء (Instance) یک نمونه (row) در این مثال، هر سطر
# رو نشون می‌ده، مثل سن یا نمره (Attribute) یا صفت (Feature) هم یک ویژگی (column) هر ستون
# گفته می‌شه **(Data Matrix) به این ساختار جدولی، **ماتریس داده
data = {
    'ID': [1, 2, 3, 4],
    'Feature1': [10, 15, 20, 25],
    'Feature2': [0.5, 1.2, 0.8, 1.0]
}

# جدید می‌سازیم 'DataFrame' با استفاده از داده‌های بالا، به
# 'DataFrame' همون ماتریس داده‌ست که
df = pd.DataFrame(data)

# به تابع ساده‌اس که هر چیزی رو بهمون نشون می‌ده 'print'
# رو چاپ می‌کنیم تا ببینیم ماتریس داده چه شکلیه 'DataFrame' اینجا
print("ماتریس داده:")
print(df)
```

ماتریس داده:

	ID	Feature1	Feature2
0	1	10	0.5
1	2	15	1.2
2	3	20	0.8
3	4	25	1.0

```
In [15]: from scipy.spatial.distance import pdist, squareform
import pandas as pd

# این قسمت اول کده. دو تا کتابخونه اصلی رو وارد می‌کنیم
# 'scipy.spatial.distance' می‌شه استفاده می‌شه
# 'pandas' که ابزار اصلی ما برای کار با داده‌هاست

# این دیکشنری حاوی داده‌های نمونه ماست که یک ماتریس داده رو تشکیل می‌ده
# 'ID' و 'Feature1' یک شناسه‌ست و 'Feature2'
data = {
    'ID': [1, 2, 3, 4],
    'Feature1': [10, 15, 20, 25],
    'Feature2': [0.5, 1.2, 0.8, 1.0]
}

# جدید می‌سازیم 'DataFrame' با استفاده از داده‌های بالا، به
df = pd.DataFrame(data)

# رو محاسبه می‌کنه **(Dissimilarity Matrix) این بخش، **ماتریس عدم تشابه
# ماتریس عدم تشابه، فاصله بین هر جفت از نمونه‌ها رو نشون می‌ده. هرچی فاصله بیشتر باشه، عدم تشابه یا تفاوتشون هم بیشتره
# تابع اصلی برای محاسبه فاصله‌هاست 'pdist()'
# یا معیار فاصله استفاده می‌کنیم که همون فاصله اقلیدسی (فاصله مستقیم بین دو نقطه) هست 'metric' به عنوان 'euclidean' ما از
# هم به تابع می‌گه که فقط از این دو ستون برای محاسبه فاصله استفاده کنه 'df[['Feature1', 'Feature2']]
# هم فاصله‌های محاسبه شده رو به شکل یک ماتریس مربع درمیاره که خوندنش راحت‌تره 'squareform()'
dissimilarity_matrix = squareform(pdist(df[['Feature1', 'Feature2']], metric='euclidean'))

# در نهایت، ماتریس عدم تشابه رو چاپ می‌کنیم
# 'pd.DataFrame()' بهمون کمک می‌کنه که ماتریس رو تو قالب یه جدول مرتب با برچسب‌های
print("ماتریس عدم تشابه:")
print(pd.DataFrame(dissimilarity_matrix, index=df['ID'], columns=df['ID']))
```

ماتریس عدم تشابه:

ID	1	2	3	4
ID				
1	0.000000	5.048762	10.004499	15.008331
2	5.048762	0.000000	5.015974	10.002000
3	10.004499	5.015974	0.000000	5.003998
4	15.008331	10.002000	5.003998	0.000000



توضیحات مثال کد پایتون

در این کد، تابع **pdist** از کتابخانه **SciPy** فواصل اقلیدسی (Euclidean distances) بین جفت‌های نمونه را بر اساس ویژگی‌های انتخاب‌شده محاسبه می‌کند. تابع **squareform** نیز ماتریس فاصله فشرده‌شده را به یک ماتریس مربعی تبدیل می‌کند.

ماتریس‌های داده یک نمایش ساختاریافته از داده‌های خام ارائه می‌دهند، در حالی که ماتریس‌های عدم تشابه، عدم تشابه جفت‌ها را نشان می‌دهند که در کاربردهای مختلف داده‌کاوی مانند **خوشه‌بندی (clustering)**، **طبقه‌بندی (classification)** و **جستجوی شباهت (similarity search)** مفید هستند.

2.3.2. Proximity measures for nominal attributes



معیارهای نزدیکی برای ویژگی‌های اسمی

معیارهای نزدیکی برای ویژگی‌های اسمی (Proximity measures for nominal attributes) برای تعیین میزان تشابه یا عدم تشابه بین نمونه‌ها، هنگامی که با ویژگی‌های کیفی یا اسمی سروکار داریم، استفاده می‌شوند. برخلاف ویژگی‌های عددی، ویژگی‌های اسمی ترتیب طبیعی ندارند، که باعث می‌شود معیارهای فاصله سنتی برای آن‌ها نامناسب باشند. معیارهای نزدیکی این مشکل را با تمرکز بر تطابق یا عدم تطابق بین مقادیر کیفی حل می‌کنند.

مثال معیار نزدیکی: شباهت جاکارد

توضیح:

شباهت جاکارد (**Jaccard Similarity**)، میزان شباهت بین دو مجموعه را با تقسیم اندازه اشتراک آن‌ها بر اندازه اجتماعشان اندازه‌گیری می‌کند. در زمینه ویژگی‌های اسمی، هر مقدار ویژگی به عنوان مجموعه‌ای از مقادیر دودویی (Binary) در نظر گرفته می‌شود (۱ اگر ویژگی وجود داشته باشد، ۰ در غیر این صورت).

```
In [20]: from sklearn.metrics import jaccard_score
import pandas as pd
```

```
# این بخش اول کده. ما دو تا کتابخونه رو وارد می‌کنیم
# که به ابزار خیلی پرکاربرده برای کار کردن با داده‌ها، به خصوص داده‌های جدولی 'pandas'
# هست که برای محاسبه ضریب ژاکارد استفاده می‌شه 'sklearn.metrics' هم یک تابع از کتابخونه 'jaccard_score'

# حاوی داده‌های نمونه ماست (dictionary) این به دیکشنری
# هستن؛ یعنی فقط اسم یا دسته هستن و هیچ ترتیب مشخصی بینشون وجود نداره (Nominal) این داده‌ها از نوع اسمی
# فقط دو دسته متفاوت از رنگ هستن 'Blue' و 'Red' مثلاً
data = {
    'Instance1': ['Red', 'Square', 'Large'],
    'Instance2': ['Blue', 'Circle', 'Small']
}

# برای اینکه بتونیم ضریب ژاکارد رو حساب کنیم، باید داده‌های اسمی رو به داده‌های باینری (صفر و یک) تبدیل کنیم
# این کار رو برامون انجام می‌ده 'pandas' از کتابخونه 'pd.get_dummies()' تابع
# تو داده‌های اسمی، به ستون جدید می‌سازه (Unique) این تابع برای هر مقدار منحصر به فرد
# اگه اون مقدار وجود داشته باشه، عدد ۱ و اگه نباشه، عدد ۰ رو قرار می‌ده
# هم می‌گن **One-Hot Encoding** به این فرآیند
binary_data = pd.get_dummies(pd.DataFrame(data))

# نهایی که به صورت باینری تبدیل شده، نمایش داده می‌شه 'DataFrame'، با اجرای این خط
# شباهتشون بررسی بشه، 'jaccard_score' این داده‌ها حالا آماده‌ان تا با استفاده از
binary_data
```

```
Out[20]:
```

	Instance1_Large	Instance1_Red	Instance1_Square	Instance2_Blue	Instance2_Circle	Instance2_Small
0	False	True	False	True	False	False
1	False	False	True	False	True	False
2	True	False	False	False	False	True

```
In [22]: # رو انجام می‌ده (Jaccard Similarity) این بخش، محاسبه ضریب ژاکارد
# ضریب ژاکارد به معیار برای سنجش شباهت بین دو مجموعه هست
# رو انتخاب می‌کنن 'DataFrame' سطر اول و دوم 'binary_data.iloc[1]' و 'binary_data.iloc[0]'
# داده می‌شن تا شباهتشون محاسبه بشه 'jaccard_score' این مقادیر باینری به تابع
# اگه دو نمونه کاملاً مشابه باشن، مقدار ۱ و اگه هیچ شباهتی نداشته باشن، مقدار ۰ رو برمی‌گردونه

jaccard_similarity = jaccard_score(binary_data.iloc[0], binary_data.iloc[1])

# اینجا هم ضریب ژاکارد به نمونه با خودش حساب شده که همیشه باید برابر با ۱ باشه

jaccard_similarity_1 = jaccard_score(binary_data.iloc[0], binary_data.iloc[0])

print(f"Jaccard Similarity: {jaccard_similarity}")
print(f"Jaccard Similarity: {jaccard_similarity_1}")
```

```
Jaccard Similarity: 0.0
Jaccard Similarity: 1.0
```

توضیحات مثال کد پایتون

در این مثال، ویژگی‌های اسمی ('Red', 'Square', 'Large') و ('Blue', 'Circle', 'Small') با استفاده از one-hot encoding به بردارهای دودویی (Binary Vectors) تبدیل می‌شوند. سپس، شباهت جاکارد (**Jaccard Similarity**) با استفاده از تابع

`jaccard_score` از کتابخانه `scikit-learn` محاسبه می‌شود.

معیارهای نزدیکی برای ویژگی‌های اسمی، برای خوشه‌بندی، طبقه‌بندی و سایر وظایف داده‌کاوی که شامل داده‌های کیفی هستند، ضروری‌اند. این مثال عملی کاربرد جاکارد را برای کمی‌سازی شباهت بین نمونه‌ها با ویژگی‌های اسمی نشان می‌دهد.

2.3.3. Proximity measures for binary attributes

✓ معیارهای نزدیکی برای ویژگی‌های دودویی

معیارهای نزدیکی برای ویژگی‌های دودویی (Proximity measures for binary attributes) هنگام کار با مجموعه‌داده‌هایی که شامل ویژگی‌های دودویی (0/1) هستند، به کار گرفته می‌شوند. این معیارها تشابه یا عدم تشابه بین نمونه‌ها را بر اساس حضور یا عدم حضور ویژگی‌های دودویی خاص ارزیابی می‌کنند.

✓ مثال معیار نزدیکی: فاصله همینگ

توضیح:

فاصله همینگ (Hamming Distance) عدم تشابه بین دو رشته دودویی را با شمارش تعداد موقعیت‌هایی که بیت‌های متناظر در آن‌ها متفاوت هستند، اندازه‌گیری می‌کند.

In [24]:

```
import numpy as np
from scipy.spatial.distance import hamming

# این قسمت اول کده. دو تا کتابخانه رو وارد می‌کنیم
# که به ابزار خیلی پرکاربرد برای کار با آرایه‌ها و محاسبات ریاضیه 'numpy'
# استفاده می‌شه (Hamming Distance) هست و برای محاسبه فاصله همینگ 'scipy.spatial.distance' که تابعی از 'hamming'

# با داده‌های نمونه ماست (dictionary) این یه دیکشنری
# هستن، یعنی فقط مقادیر ۰ و ۱ رو دارن (Binary) این داده‌ها از نوع باینری
# این مقادیر می‌تونن نشون‌دهنده حضور یا عدم حضور یک ویژگی خاص باشن
data = {
    'Instance1': [1, 0, 1, 0],
    'Instance2': [0, 1, 1, 0]
}

# محاسبه کنیم 'hamming' برای اینکه بتونیم فاصله همینگ رو با تابع
# تبدیل کنیم 'numpy' باید داده‌ها رو به آرایه‌های
# این خط دو تا آرایه می‌سازه که هر کدوم شامل مقادیر یک نمونه هستن
binary_data = np.array([data['Instance1'], data['Instance2']])

# رو محاسبه می‌کنه (Hamming Distance)** این بخش، **فاصله همینگ
# فاصله همینگ، تعداد موقعیت‌هایی رو می‌شماره که مقادیر دو نمونه در اون‌ها با هم فرق دارن
# مثلاً اگه دو رشته '1010' و '0110' رو مقایسه کنیم، می‌بینیم که فقط در دو موقعیت (اول و دوم) با هم فرق دارن
# بنابراین، فاصله همینگشون برابر با ۲ هست
# این فاصله رو برامون حساب می‌کنه 'hamming(binary_data[0], binary_data[1])'
# نتیجه یه عدد اعشاریه، چون تابع مقدار رو به صورت میانگین اختلاف برمی‌گردونه
hamming_distance = hamming(binary_data[0], binary_data[1])

print(f"Hamming Distance: {hamming_distance}")
```

Hamming Distance: 0.5

توضیحات مثال فاصله همینگ

در این مثال، نمونه‌ها با بردارهای دودویی $[1, 0, 1, 0]$ و $[0, 1, 1, 0]$ نمایش داده شده‌اند. فاصله همینگ بین این بردارها با استفاده از تابع `hamming` از کتابخانه **SciPy** محاسبه می‌شود.

معیارهای نزدیکی برای ویژگی‌های دودویی، برای وظایفی مانند خوشه‌بندی، تشخیص الگو و طبقه‌بندی، هنگام کار با مجموعه‌داده‌های متشکل از ویژگی‌های دودویی بسیار حیاتی هستند. مثال فاصله همینگ نشان می‌دهد که چگونه می‌توان عدم تشابه بین نمونه‌ها را بر اساس ویژگی‌های دودویی کمی‌سازی کرد.

2.3.4. Dissimilarity of numeric data: Minkowski distance

فاصله مینکوفسکی

فاصله مینکوفسکی (Minkowski distance) یک معیار عدم تشابه است که برای ارزیابی میزان تشابه یا عدم تشابه بین دو نقطه در یک فضای چندبعدی استفاده می‌شود. این معیار یک تعمیم برای سایر معیارهای فاصله، از جمله **فاصله اقلیدسی (Euclidean distance)** و **فاصله منهتن (Manhattan distance)**، محسوب می‌شود و با فرمول زیر تعریف می‌شود:

$$D(x, y) = \left(\sum_{i=1}^n |x_i - y_i|^p \right)^{\frac{1}{p}}$$

که در آن **x** و **y** بردارهای نمایانگر نقاط داده عددی هستند، **n** تعداد ابعاد است، و **p** پارامتری است که مرتبه فاصله مینکوفسکی را تعیین می‌کند. هنگامی که **p=1** باشد، فاصله مینکوفسکی معادل فاصله منهتن است، و زمانی که **p=2** باشد، معادل فاصله اقلیدسی است.

```
In [27]: from scipy.spatial.distance import minkowski
import numpy as np
```

```
# این قسمت اول کده. ما دو تا کتابخانه رو وارد می‌کنیم
# که برای محاسبه فاصله‌ها استفاده می‌شه 'scipy.spatial.distance'
# که برای کار با آرایه‌ها و محاسبات ریاضیه 'numpy'

# این خطوط داده‌های نمونه ما هستن که به صورت لیست تعریف شدن
# این‌ها دو نمونه با ویژگی‌های عددی هستن که می‌خوایم فاصله‌شون رو از هم محاسبه کنیم
data1 = [2, 3, 5, 7]
data2 = [1, 4, 6, 8]

# محاسبه می‌کنه p=2 رو با ** (Minkowski Distance) این بخش **فاصله مینکوفسکی
# فاصله مینکوفسکی یه معیار کلی برای محاسبه فاصله بین دو نقطه در فضای چندبعدیه
# می‌شه ** (Euclidean Distance) برابر با ۲ باشه، فاصله مینکوفسکی دقیقاً برابر با **فاصله اقلیدسی p وقتی
# فاصله اقلیدسی، کوتاه‌ترین فاصله بین دو نقطه در یک خط مستقیمه
minkowski_distance = minkowski(data1, data2, p=2)
print(f"(p=2): فاصله مینکوفسکی {minkowski_distance}")

# محاسبه می‌کنه p=1 این قسمت **فاصله مینکوفسکی** رو با
# می‌شه ** (Manhattan Distance) برابر با ۱ باشه، فاصله مینکوفسکی برابر با **فاصله منهتن p وقتی
# فاصله منهتن، جمع قدر مطلق تفاوت هر ویژگیه، مثل فاصله‌ای که در خیابان‌های یک شهر (مثل منهتن) طی می‌شه
manhattan_distance = minkowski(data1, data2, p=1)
print(f"(p=1): فاصله مینکوفسکی {manhattan_distance}")
```

```
(p=2): فاصله مینکوفسکی 2.0
(p=1): فاصله مینکوفسکی 4.0
```

توضیحات مثال کد پایتون

در این مثال، فاصله مینکوفسکی بین دو بردار عددی [۲,۳,۵,۷] و [۱,۴,۶,۸] با استفاده از فاصله اقلیدسی (p=2) و فاصله منهتن (p=1) محاسبه می‌شود. برای این محاسبه از تابع minkowski از کتابخانه **SciPy** استفاده شده است. فاصله مینکوفسکی یک معیار منعطف است که بر اساس مقدار انتخاب‌شده برای **p**، با سناریوهای مختلف سازگار می‌شود. این معیار در وظایف داده‌کاوی مختلفی مانند خوشه‌بندی، طبقه‌بندی و تشخیص نقاط پرت کاربرد دارد، به خصوص زمانی که با مجموعه داده‌های حاوی ویژگی‌های عددی سروکار داریم.

2.3.5. Proximity measures for ordinal attributes

معیارهای نزدیکی برای ویژگی‌های ترتیبی

معیارهای نزدیکی برای ویژگی‌های ترتیبی، هنگام کار با مجموعه داده‌هایی که شامل ویژگی‌هایی با ترتیب یا رتبه‌بندی ذاتی هستند، استفاده می‌شوند. برخلاف ویژگی‌های اسمی، ویژگی‌های ترتیبی دارای ترتیب معناداری هستند، اما ممکن است فواصل بین مقادیر آن‌ها یکنواخت نباشد.

مثال معیار نزدیکی: ضریب همبستگی رتبه‌ای اسپیرمن

توضیح:

ضریب همبستگی رتبه‌ای اسپیرمن (Spearman Rank Correlation Coefficient) قدرت و جهت رابطه یکنواخت (monotonic) بین دو متغیر ترتیبی را اندازه‌گیری می‌کند. این معیار به جای مقادیر واقعی، بر اساس رتبه‌های (ranks) مقادیر محاسبه می‌شود.

```
In [28]: import numpy as np
from scipy.stats import spearmanr

# این قسمت اول کده. ما دو تا کتابخونه رو وارد می‌کنیم
# که برای کار با آرایه‌ها و محاسبات ریاضیه 'numpy'
# استفاده می‌شه (Spearman Rank Correlation Coefficient) هست و برای محاسبه ضریب همبستگی رتبه‌ای اسپیرمن 'scipy.stats' که تابعی از 'spearmanr'

# با داده‌های نمونه ماست (dictionary) این په دیکشنری
# هستن، یعنی فقط ترتیشون مهمه (Ordinal) این داده‌ها از نوع ترتیبی
# مقدار ۲ از همه کوچیکتره، پس رتبه اول رو می‌گیره، 'Instance1' در نظر گرفته می‌شه. مثلاً تو (Rank) به جای خود مقادیر، رتبه‌شون
data = {
    'Instance1': [2, 3, 1, 4],
    'Instance2': [1, 4, 2, 3]
}

# این بخش، **ضریب همبستگی رتبه‌ای اسپیرمن** رو محاسبه می‌کنه
# این ضریب، قدرت و جهت رابطه بین دو متغیر ترتیبی رو نشون می‌ده
# اگه مقدارش نزدیک به ۱ باشه، یعنی هر دو متغیر با هم رشد می‌کنن
# اگه نزدیک به -۱ باشه، یعنی یکی رشد می‌کنه و اون یکی کم می‌شه
# تابع اصلی برای این محاسبه‌اس 'spearmanr()'
# که ما فقط ضریب رو نگه می‌داریم 'p-value' این تابع دو مقدار رو برمی‌گردونه: ضریب همبستگی و مقدار
spearman_corr, _ = spearmanr(data['Instance1'], data['Instance2'])

print(f"ضریب همبستگی رتبه‌ای اسپیرمن: {spearman_corr}")
```

ضریب همبستگی رتبه‌ای اسپیرمن: 0.6000000000000001

توضیحات مثال کد پایتون

در این مثال، نمونه‌ها با ویژگی‌های ترتیبی به صورت بردارهای [۲,۳,۱,۴] و [۱,۴,۲,۳] نمایش داده شده‌اند. ضریب همبستگی رتبه‌ای اسپیرمن با استفاده از تابع **spearmanr** از کتابخانه **SciPy** محاسبه می‌شود. معیارهای نزدیکی برای ویژگی‌های ترتیبی، هنگام کار با داده‌هایی که دارای ترتیب معنادار هستند اما فاقد مقیاس عددی مشخص می‌باشند، اهمیت زیادی پیدا می‌کنند. مثال ضریب همبستگی رتبه‌ای اسپیرمن نشان می‌دهد که چگونه می‌توان رابطه یکنواخت بین متغیرهای ترتیبی را ارزیابی کرد.

2.3.6. Dissimilarity for attributes of mixed types

مدیریت عدم تشابه برای ویژگی‌های با نوع ترکیبی

مدیریت عدم تشابه برای ویژگی‌های با نوع ترکیبی یک چالش رایج در داده‌کاوی است، زیرا مجموعه داده‌ها اغلب شامل ترکیبی از ویژگی‌های عددی (numeric)، کیفی (categorical) و ترتیبی (ordinal) هستند. استفاده از معیارهای عدم تشابه مناسب که بتوانند ماهیت متنوع این انواع ویژگی‌ها را در خود جای دهند، بسیار حیاتی است.

مثال معیار عدم تشابه: فاصله گاور (Gower's Distance)

توضیح:

فاصله گاور (Gower's Distance) یک معیار عدم تشابه است که برای مجموعه داده‌های با انواع ویژگی‌های ترکیبی طراحی شده

است. این معیار عدم تشابه بین دو نمونه را با در نظر گرفتن انواع ویژگی‌ها و اعمال معیارهای مناسب، مانند فاصله اقلیدسی

برای ویژگی‌های عددی، شباهت جاکارد برای ویژگی‌های دودویی و ضریب تطابق ساده برای ویژگی‌های اسمی، محاسبه می‌کند.

In [29]:

```
import numpy as np
import pandas as pd
from scipy.spatial.distance import pdist, squareform
from sklearn.preprocessing import StandardScaler

# این قسمت اول کده. ما چند تا کتابخونه رو وارد می‌کنیم
# برای کار با داده‌های جدولی 'pandas'، برای کار با آرایه‌ها 'numpy'
# برای محاسبه فاصله‌ها 'scipy' از 'squareform' و 'pdist'،
# برای استانداردسازی داده‌های عددی 'sklearn' از 'StandardScaler' و

# این دیکشنری حاوی داده‌های نمونه ماست که انواع مختلفی از ویژگی‌ها رو داره
# ویژگی‌های عددی مثل سن یا قد 'Numeric':
# ویژگی‌های باینری مثل 'بله' یا 'خیر' 'Binary':
# ویژگی‌های اسمی مثل رنگ یا شهر 'Nominal':
# ویژگی‌های ترتیبی مثل سطح تحصیلات 'Ordinal':
data = {
    'Numeric': [2.5, 1.8, 3.2, 4.0],
    'Binary': [1, 0, 1, 1],
    'Nominal': ['A', 'B', 'A', 'C'],
    'Ordinal': [2, 1, 3, 2]
}

# می‌کنه (Standardize) این بخش، داده‌های عددی رو استانداردسازی
# استانداردسازی یعنی داده‌ها رو به صورتی تغییر می‌دیم که میانگینشون صفر و انحراف معیارشون یک بشه
# این کار برای مقایسه داده‌ها با مقیاس‌های مختلف خیلی مهمه
numeric_data = np.array([data['Numeric']]).T
numeric_data = StandardScaler().fit_transform(numeric_data)

# تبدیل می‌کنه تا بتونیم باهاش کار کنیم 'DataFrame' این قسمت ویژگی‌های باینری رو به یک
# این داده‌ها به صورت طبیعی باینری هستن و نیازی به تبدیل ندارن
binary_data = pd.get_dummies(pd.DataFrame(data['Binary'], columns=['Binary']))

# این بخش، ویژگی‌های اسمی رو به داده‌های باینری تبدیل می‌کنه
# هست که قبلاً دیدیم و برای مقایسه داده‌های اسمی ضروریه **One-Hot Encoding** این کار همون
nominal_data = pd.get_dummies(pd.DataFrame(data['Nominal'], columns=['Nominal']))

# حالا همه انواع داده‌ها (که همشون عددی یا باینری شدن) رو با هم ترکیب می‌کنیم
# داده‌ها رو کنار هم می‌چینه و یک آرایه بزرگ می‌سازه 'np.concatenate()'
# بهش می‌گه که این ترکیب رو ستون به ستون انجام بده 'axis=1'
mixed_data = np.concatenate((numeric_data, binary_data, nominal_data, np.array([data['Ordinal']]).T), axis=1)

# رو محاسبه می‌کنه **Gower's Distance** این بخش **فاصله گاور**
# فاصله گاور یک معیار فاصله مخصوصه برای داده‌هایی که انواع مختلفی دارن
# این معیار، فاصله بین داده‌های عددی، اسمی و باینری رو به صورت همزمان حساب می‌کنه
# استفاده شده، چون قبلاً داده‌ها رو آماده و استاندارد کردیم 'metric='euclidean'' با 'pdist' در اینجا، از
gower_distance = squareform(pdist(mixed_data, metric='euclidean'))
print("ماتریس فاصله گاور")
print(gower_distance)
```

ماتریس فاصله گاور:

```
[[0.          2.17595275  1.31710682  2.31817722]
 [2.17595275  0.          3.15263089  3.35525438]
 [1.31710682  3.15263089  0.          1.98989952]
 [2.31817722  3.35525438  1.98989952  0.          ]]
```

توضیحات مثال کد پایتون

در این مثال، **فاصله گاور** برای یک مجموعه داده با ویژگی‌های عددی (numeric)، دودویی (binary)، اسمی (nominal) و ترتیبی

(ordinal) محاسبه می‌شود. ویژگی‌های عددی استاندارد می‌شوند، ویژگی‌های دودویی و اسمی به مقادیر دودویی تبدیل می‌گردند،

و همه انواع ویژگی‌ها در یک ماتریس واحد ترکیب می‌شوند. سپس، تابع **pdist** از **SciPy** برای محاسبه فواصل اقلیدسی

جفت‌ها و تابع **squareform** برای تبدیل ماتریس فاصله فشرده‌شده به ماتریس مربعی استفاده می‌شود.

فاصله گاور یک معیار منعطف برای سنجش عدم تشابه در مجموعه داده‌های با انواع ویژگی‌های ترکیبی است و راه‌حلی جامع برای

کار با ویژگی‌های متنوع داده ارائه می‌دهد.

2.3.7. Cosine similarity

شباهت کسینوسی

شباهت کسینوسی (Cosine similarity) معیاری است که برای اندازه‌گیری شباهت بین دو بردار غیرصفر در یک فضای ضرب

داخلی (inner product space) استفاده می‌شود. در زمینه داده‌کاوی، این معیار اغلب برای ارزیابی شباهت بین اسناد در پردازش

زبان طبیعی یا مقایسه بردارهای ویژگی در سیستم‌های توصیه‌گر به کار می‌رود. شباهت کسینوسی در بازه -1 تا 1 قرار دارد، که در

آن 1 نشان‌دهنده شباهت کامل، 0 نشان‌دهنده عدم شباهت و -1 نشان‌دهنده عدم تشابه کامل است.

```
In [30]: from sklearn.feature_extraction.text import CountVectorizer
from sklearn.metrics.pairwise import cosine_similarity
import numpy as np
```

```
# وارد می‌کنیم 'sklearn' این قسمت اول کده. ما دو تا ابزار مهم رو از کتابخونه
# استفاده می‌شه (vector) که برای تبدیل متن به بردار 'CountVectorizer'
# که برای محاسبه شباهت بین دو بردار از داده‌های متنی استفاده می‌شه 'cosine_similarity'
```

```
# متنی ماست (documents) حاوی سندهای (list) این لیست
# در این لیست، یک سند جداگانه در نظر گرفته می‌شه (string) هر رشته
documents = [
    "Data mining is an exciting field of study.",
    "Machine learning techniques are used for data analysis.",
    "Python programming is widely used in data science applications."
]
```

```
# 'Bag-of-Words' این بخش، داده‌های متنی رو به یک نمایش عددی تبدیل می‌کنه که بهش می‌گن
# 'CountVectorizer' هر سطرش یک سند و هر ستونش یک کلمه‌اس
# مقادیر داخل ماتریس، تعداد تکرار هر کلمه تو اون سند هستن
vectorizer = CountVectorizer()
document_matrix = vectorizer.fit_transform(documents)
```

```
# رو محاسبه می‌کنیم **(Cosine Similarity) حالا **تشابه کسینوسی
# تشابه کسینوسی، شباهت جهت‌گیری بین دو بردار رو می‌سنجه، نه اندازه‌شون رو
# این معیار، میزان شباهت معنایی بین سندها رو نشون می‌ده
# تابع اصلی برای این محاسبه‌اس 'cosine_similarity()'
# نتیجه یک ماتریس مربعی که شباهت هر سند رو با بقیه سندها نشون می‌ده
# مقادیر این ماتریس بین 0 و 1 هستن
cosine_similarities = cosine_similarity(document_matrix, document_matrix)
```

```
# در نهایت، ماتریس تشابه کسینوسی رو چاپ می‌کنیم
print("ماتریس تشابه کسینوسی:")
print(cosine_similarities)
```

ماتریس تشابه کسینوسی:

```
[[1.          0.125      0.23570226]
 [0.125       1.         0.23570226]
 [0.23570226 0.23570226 1.         ]]
```



توضیحات مثال کد پایتون

در این مثال، از `CountVectorizer` از کتابخانه `scikit-learn` برای تبدیل مجموعه‌ای از اسناد متنی به یک ماتریس از

تعداد توکن‌ها (نمایش bag-of-words) استفاده می‌شود. سپس، تابع `cosine_similarity` شباهت کسینوسی را بین

بردارهای اسناد محاسبه می‌کند.

شباهت کسینوسی به خصوص در سناریوهایی مفید است که در آن‌ها بزرگی (magnitude) بردارها اهمیت چندانی ندارد و تمرکز

بر جهت بردارها است. این معیار به طور گسترده در تحلیل متن، ارزیابی اسناد و سیستم‌های فیلترینگ مشارکتی استفاده

می‌شود. مثال ارائه‌شده نحوه محاسبه شباهت کسینوسی برای مجموعه‌ای از اسناد متنی را نشان می‌دهد.

2.3.8. Measuring similar distributions: the Kullback-Leibler divergence



واگرایی کولبک-لایبлер

واگرایی کولبک-لایبлер (KL Divergence) معیاری است که نشان می‌دهد یک توزیع احتمال چقدر از یک توزیع احتمال ثانویه و

مورد انتظار منحرف می‌شود. در زمینه داده‌کاوی، اغلب برای کمی‌سازی تفاوت یا شباهت بین دو توزیع احتمال استفاده می‌شود.

واگرایی KL یک معیار فاصله واقعی نیست، زیرا متقارن نمی‌باشد و از نامساوی مثلث پیروی نمی‌کند.

```
In [31]: import numpy as np
from scipy.stats import entropy
```

:این قسمت اول کده. ما دو تا ابزار رو وارد می‌کنیم #

```
# که برای کار با آرایه‌ها و داده‌های عددی 'numpy'
# استفاده می‌شه (KL Divergence) هست و برای محاسبه واگرایی کولیک-لایپلر 'scipy.stats' که تابعی از 'entropy'
```

```
# رو نشون می‌دن (Probability Distribution) این خطوط، دو تا توزیع احتمال
# هر توزیع نشون می‌ده که احتمال هر رویداد چقدره
# می‌گه که احتمال رویداد اول ۰.۲، دوم ۰.۳ و سوم ۰.۵ هست 'distribution1' مثلاً
distribution1 = np.array([0.2, 0.3, 0.5])
distribution2 = np.array([0.1, 0.6, 0.3])
```

```
# این بخش، **واگرایی کولیک-لایپلر** رو محاسبه می‌کنه
# یک معیار برای سنجش تفاوت بین دو توزیع احتماله (Divergence) این واگرایی
# هر چقدر مقدارش بیشتر باشه، یعنی دو توزیع با هم متفاوت‌تر هستن
# برابر نیست  $KL(q, p)$  با  $KL(p, q)$  واگرایی کولیک-لایپلر نامتقارنه، یعنی
# تابعیه که این مقدار رو حساب می‌کنه. تو این تابع، توزیع اول رو به عنوان توزیع اصلی و توزیع دوم رو به عنوان توزیع مرجع در نظر می‌گیره 'entropy()'
kl_divergence = entropy(distribution1, distribution2)
```

```
# در نهایت، نتیجه محاسبه رو چاپ می‌کنیم
print(f"واگرایی کولیک-لایپلر: {kl_divergence}")
```

واگرایی کولیک-لایپلر : 0.18609809382700082

توضیحات مثال کد پایتون

در این مثال، دو توزیع احتمال با آرایه‌های `distribution1` و `distribution2` نمایش داده شده‌اند. تابع `entropy` از کتابخانه **SciPy**، واگرایی **KL Divergence** را بین این توزیع‌ها محاسبه می‌کند. واگرایی **KL Divergence** اغلب در نظریه اطلاعات و آمار برای اندازه‌گیری تفاوت بین دو توزیع احتمال استفاده می‌شود. این معیار در زمینه‌های مختلفی از جمله پردازش زبان طبیعی، یادگیری ماشین و تشخیص الگو کاربرد دارد.

2.3.9. Capturing hidden semantics in similarity measures

درک معنای پنهان در معیارهای شباهت

درک معنای پنهان (Capturing hidden semantics) در معیارهای شباهت شامل یافتن الگوها، روابط یا شباهت‌های معنادار در داده‌ها است که ممکن است در نگاه اول آشکار نباشند. این فرآیند اغلب به تکنیک‌های پیشرفته‌ای نیاز دارد که فراتر از معیارهای فاصله یا شباهت ساده عمل می‌کنند. برای درک و بهره‌برداری از معنای پنهان در داده‌ها، از روش‌هایی مانند Word Embeddings (تعبیه‌های کلمه)، شباهت معنایی (Semantic Similarity) یا مدل‌های پیشرفته شبکه عصبی استفاده می‌شود.

In []: `pip install gensim`

In [36]: `from gensim.models import Word2Vec
from sklearn.metrics.pairwise import cosine_similarity
import numpy as np`

```
# این قسمت اول کده. دو تا ابزار مهم رو از کتابخونه‌های مختلف وارد می‌کنیم
# استفاده می‌شه (word embeddings) که برای ساختن بردارهای کلمات 'gensim' از 'Word2Vec'
# که برای محاسبه شباهت بین دو بردار استفاده می‌شه 'sklearn' از 'cosine_similarity'
```

```
# متنی ماست (documents) حاوی سندهای (list) این لیست
# در این لیست، یک سند جداگانه در نظر گرفته می‌شه (string) هر رشته
documents = [
    "Data mining is an exciting field of study.",
    "Machine learning techniques are used for data analysis.",
    "Python programming is widely used in data science applications."
]
```

```
# استفاده کنیم، باید هر سند رو به کلماتش تقسیم کنیم 'Word2Vec' برای اینکه بتونیم از
# نامیده می‌شه ** (Tokenize) این فرآیند **توکنایز کردن
# این خط، هر سند رو به یه لیست از کلمات تبدیل می‌کنه
tokenized_documents = [doc.split() for doc in documents]
```

```
# رو روی داده‌های توکنایز شده، آموزش می‌ده 'Word2Vec' این بخش، یک مدل
# کلمات رو به بردارهای عددی تبدیل می‌کنه، به طوری که کلماتی که معنی مشابهی دارن 'Word2Vec'
# بردارهای نزدیک به همی داشته باشن
# طول هر بردار رو ۱۰ می‌ذاره 'vector_size=10'
# به مدل می‌گه که ۳ کلمه قبل و بعد از کلمه فعلی رو نگاه کنه 'window=3'
# کلماتی که حداقل یک بار تکرار شدن رو در نظر می‌گیره 'min_count=1'
model = Word2Vec(tokenized_documents, vector_size=10, window=3, min_count=1, workers=4)
```

```
# حالا با استفاده از مدل آموزش‌دیده، **شباهت کسینوسی** رو محاسبه می‌کنیم
# رو پیدا می‌کنن 'mining' و 'data' بردارهای مربوط به کلمات 'model.wv['mining']' و 'model.wv['data']'
# هم شباهت جهت‌گیری این دو بردار رو حساب می‌کنه 'cosine_similarity()' تابع
# یک اصطلاح مشترکه، انتظار داریم این دو کلمه شباهت زیادی به هم داشته باشن 'data mining' چون
embedding_similarity = cosine_similarity([model.wv['data']], [model.wv['mining']])
```

```
# در نهایت، نتیجه محاسبه رو چاپ می‌کنیم
print(f"Cosine Similarity using Word Embeddings: {embedding_similarity[0][0]}")
```

Cosine Similarity using Word Embeddings: -0.38179394602775574

توضیحات مثال کد پایتون

در این مثال، کتابخانه Gensim برای آموزش یک مدل Word2Vec بر روی یک مجموعه داده کوچک از اسناد متنی استفاده می‌شود. تعبیه‌های کلمه (Word embeddings) روابط معنایی بین کلمات را درک می‌کنند و سپس از شباهت کسینوسی (cosine similarity) برای اندازه‌گیری شباهت بین بردارهای کلمه دو کلمه ('data' و 'mining') استفاده می‌شود. درک معنای پنهان اغلب شامل بهره‌گیری از تکنیک‌های پیشرفته‌ای مانند تعبیه‌های کلمه، شبکه‌های عصبی یا مدل‌های یادگیری عمیق است. این روش‌ها به‌ویژه در کاربردهایی مانند پردازش زبان طبیعی، سیستم‌های توصیه‌گر و تحلیل تصاویر که در آن‌ها درک معنی یا مفهوم زیربنایی بسیار حیاتی است، مفید هستند.

Section 2.4

2.4.1. Data quality measures

معیارهای کیفیت داده

معیارهای کیفیت داده (Data quality measures) برای ارزیابی اعتبار و دقت یک مجموعه داده به کار می‌روند. اطمینان از کیفیت بالای داده‌ها برای موفقیت در داده‌کاوی و تحلیل ضروری است. از معیارها و تکنیک‌های مختلفی برای ارزیابی جنبه‌های گوناگون کیفیت داده، از جمله کامل بودن (completeness)، دقت (accuracy)، سازگاری (consistency) و بهنگام بودن (timeliness) استفاده می‌شود.

مثال معیار کیفیت داده: مقادیر گمشده

توضیح:

مقادیر گمشده (Missing Values) یک مشکل رایج در مجموعه‌داده‌ها هستند و می‌توانند بر کیفیت تحلیل تأثیر بگذارند. یکی از معیارهای کیفیت داده، ارزیابی درصد مقادیر گمشده در هر ستون است.

In [37]:

```
import pandas as pd
import numpy as np

# این قسمت اول کده. ما دو تا کتابخانه رو وارد می‌کنیم
# که ابزار اصلی ما برای کار با داده‌های جدولی هست 'pandas'
# استفاده می‌شه 'None' هم برای کار با داده‌های عددی و مقادیر خاص مثل 'numpy'

# با داده‌های نمونه ماست (dictionary) این به دیکشنری
# مشخص شدن 'None' در این دیتاست، بعضی از مقادیر به صورت عمدی حذف شدن و با
# هست که باید قبل از تحلیل داده‌ها، باهاشون برخورد کرد (Missing Values) نشون‌دهنده مقادیر گمشده 'None'
data = {
    'ID': [1, 2, 3, 4, 5],
    'Feature1': [10, 15, None, 25, 20],
    'Feature2': [5, None, 8, 12, 10],
    'Feature3': [2, 4, 6, None, 8]
}

# جدید می‌سازیم 'DataFrame' با استفاده از داده‌های بالا، به
df = pd.DataFrame(data)

# این بخش، درصد مقادیر گمشده در هر ستون رو محاسبه می‌کنه
# پر می‌کنه 'False' و بقیه رو با 'True' باشه رو با 'None' جدید می‌سازه که هر مقداری که 'DataFrame' به 'df.isnull()'
# ها (یعنی مقادیر گمشده) رو تو هر ستون حساب می‌کنه 'True' مجموع 'sum()'
# رو می‌ده 'DataFrame' تعداد کل سطرهای 'len(df)'
# در نهایت، با تقسیم تعداد مقادیر گمشده بر تعداد کل سطرها و ضرب در ۱۰۰، درصد رو به دست می‌آریم
missing_percentage = (df.isnull().sum() / len(df)) * 100
print(f"درصد مقادیر گمشده:")
print(missing_percentage)
```

درصد مقادیر گمشده:
ID 0.0
Feature1 20.0
Feature2 20.0
Feature3 20.0
dtype: float64



توضیحات مثال کد پایتون

در این مثال، مجموعه داده حاوی مقادیر گمشده‌ای است که با None نمایش داده شده‌اند. این کد درصد مقادیر گمشده در هر ستون را با استفاده از تابع `isnull()` و تابع `sum()` در پانداس محاسبه می‌کند. ارزیابی کیفیت داده برای اتخاذ تصمیمات آگاهانه در طول فرآیند داده‌کاوی حیاتی است. این مثال بر مقادیر گمشده تمرکز دارد، اما سایر معیارهای کیفیت داده می‌توانند شامل بررسی نقاط پرت (outliers)، اطمینان از سازگاری در مقادیر کیفی و اعتبارسنجی داده‌ها در برابر قوانین تجاری از پیش تعریف‌شده باشند.

2.4.2. Data cleaning



پاکسازی داده (Data Cleaning)

پاکسازی داده یک گام حیاتی در مرحله پیش‌پردازش داده‌ها (data preprocessing) است که با هدف شناسایی و رفع خطاها، ناسازگاری‌ها و نادرستی‌ها در مجموعه داده انجام می‌شود. این فرآیند شامل وظایفی مانند مدیریت مقادیر گمشده (missing values)، اصلاح انواع داده‌ها، حذف موارد تکراری و رسیدگی به نقاط پرت (outliers) است. اطمینان از داده‌های تمیز و قابل اعتماد برای دستیابی به نتایج معنادار و دقیق در داده‌کاوی ضروری است.



مثال وظیفه پاکسازی داده: مدیریت مقادیر گمشده

توضیح:

یکی از وظایف رایج در پاکسازی داده، مدیریت مقادیر گمشده است. بسته به ماهیت داده‌های گمشده، می‌توان از راهکارهایی مانند تکمیل داده (imputation) یا حذف رکورد (removal) استفاده کرد.

In [38]:

```
import pandas as pd
import numpy as np

# این قسمت اول کده. ما دو تا کتابخانه رو وارد می‌کنیم
# که ابزار اصلی ما برای کار با داده‌های جدولی هست 'pandas'
# استفاده می‌شه 'None' هم برای کار با داده‌های عددی و مقادیر خاص مثل 'numpy'

# با داده‌های نمونه ماست (dictionary) این به دیکشنری
# مشخص شدن 'None' در این دیتاست، بعضی از مقادیر به صورت عمدی حذف شدن و با
# هست که باید قبل از تحلیل داده‌ها، باهاشون برخورد کرد (Missing Values) نشون‌دهنده مقادیر گمشده 'None'
data = {
    'ID': [1, 2, 3, 4, 5],
    'Feature1': [10, 15, None, 25, 20],
    'Feature2': [5, None, 8, 12, 10],
    'Feature3': [2, 4, 6, None, 8]
}

# جدید می‌سازیم 'DataFrame' با استفاده از داده‌های بالا، به
df = pd.DataFrame(data)

# این بخش، درصد مقادیر گمشده در هر ستون رو محاسبه می‌کنه
# 'df.isnull()' به 'DataFrame' جدید می‌سازه که هر مقداری که 'None' باشه رو با 'True' و بقیه رو با 'False'
# ها (یعنی مقادیر گمشده) رو تو هر ستون حساب می‌کنه 'True' مجموع 'sum()'
# رو می‌ده 'DataFrame' تعداد کل سطرهای 'len(df)'
# در نهایت، با تقسیم تعداد مقادیر گمشده بر تعداد کل سطرها و ضرب در ۱۰۰، درصد رو به دست می‌آریم
missing_percentage = (df.isnull().sum() / len(df)) * 100
print("درصد مقادیر گمشده:")
print(missing_percentage)

# بر می‌کنه (mean) این بخش، مقادیر گمشده رو با استفاده از میانگین
# یا **پر کردن داده‌ها** معروفه **imputation** این فرآیند به
# تابعیه که مقادیر گمشده رو پیدا می‌کنه و اون‌ها رو با به مقدار مشخص جایگزین می‌کنه 'df.fillna()'
# میانگین هر ستون رو حساب می‌کنه 'df.mean()'
# رو با میانگین ستون مربوطه پر می‌کنه 'None' در نهایت، این خط مقادیر
```

```
df_filled = df.fillna(df.mean())
```

```
# این خط، دیتاست نهایی رو بعد از پر کردن داده‌ها چاپ می‌کنه
print("\n دیتاست تمیز شده:")
print(df_filled)
```

درصد مقادیر گمشده:

```
ID          0.0
Feature1    20.0
Feature2    20.0
Feature3    20.0
dtype: float64
```

دیتاست تمیز شده:

	ID	Feature1	Feature2	Feature3
0	1	10.0	5.00	2.0
1	2	15.0	8.75	4.0
2	3	17.5	8.00	6.0
3	4	25.0	12.00	5.0
4	5	20.0	10.00	8.0



توضیحات مثال کد پایتون

در این مثال، مجموعه داده حاوی مقادیر گمشده‌ای است که با None نمایش داده شده‌اند. این کد از تابع **fillna** در پانداس برای جایگزینی مقادیر گمشده با مقدار میانگین هر ستون استفاده می‌کند. پاکسازی داده یک گام حیاتی برای تضمین کیفیت و قابلیت اطمینان مجموعه داده است. در حالی که این مثال بر مدیریت مقادیر گمشده تمرکز دارد، سایر وظایف پاکسازی داده ممکن است شامل حذف رکوردهای تکراری، اصلاح انواع داده‌ها و رسیدگی به نقاط پرت باشد.

2.4.3. Data integration



یکپارچه‌سازی داده

یکپارچه‌سازی داده (Data Integration) فرآیند ترکیب داده‌ها از منابع مختلف در یک نمای یکپارچه و منسجم است. این فرآیند شامل مدیریت ناسازگاری‌ها، حل تعارضات و ایجاد یک مجموعه داده ادغام‌شده است که می‌تواند برای تحلیل و داده‌کاوی استفاده شود. یکپارچه‌سازی داده هنگام کار با مجموعه‌داده‌های متنوع که ممکن است از پایگاه‌های داده، فایل‌ها یا پلتفرم‌های مختلفی به دست آمده باشند، بسیار حیاتی است.



مثال وظیفه یکپارچه‌سازی داده: ادغام مجموعه‌داده‌ها

توضیح:

یکی از وظایف رایج یکپارچه‌سازی داده، ادغام مجموعه‌داده‌ها با یک کلید یا شناسه مشترک است. این کار به ایجاد یک مجموعه داده یکپارچه کمک می‌کند که اطلاعات از چندین منبع را در بر می‌گیرد.

In [39]:

```
import pandas as pd
import numpy as np

# رو وارد می‌کنیم که برای کار با داده‌های جدولی عالیۀ 'pandas' این قسمت اول کده. ما کتابخونه

# حاوی داده‌های نمونه ما هستن (dictionary) این دو تا دیکشنری
# دارن 'ID' این داده‌ها رو به صورت دو جدول مجزا در نظر بگیرید که یک ستون مشترک به نام
data1 = {'ID': [1, 2, 3], 'Name': ['Alice', 'Bob', 'Charlie']}
data2 = {'ID': [2, 3, 4], 'Age': [25, 30, 22]}

# جدید می‌سازیم 'DataFrame' با استفاده از داده‌های بالا، دو
df1 = pd.DataFrame(data1)
df2 = pd.DataFrame(data2)

# رو با هم ترکیب می‌کنه 'DataFrame' این بخش، دو
# معروفه ** (Merge) این فرآیند به ** ادغام
# تابعیه که این کار رو انجام می‌ده 'pd.merge()'
# این دو جدول رو به هم وصل کنه 'ID' به تابع می‌گه که بر اساس ستون مشترک 'on='ID'
# هم بهش می‌گه که تمام سطرها رو از هر دو جدول نگه داره 'how='outer'
# پر می‌کنه (مقدار گمشده) 'NaN' فقط تو یه جدول باشه، مقادیر مربوط به جدول دیگه رو با 'ID' اگه یک
merged_df = pd.merge(df1, df2, on='ID', how='outer')

# ترکیب‌شده رو چاپ می‌کنیم 'DataFrame'، در نهایت
```

```
print("دیتاست ترکیب شده")
print(merged_df)
```

دیتاست ترکیب شده:

	ID	Name	Age
0	1	Alice	NaN
1	2	Bob	25.0
2	3	Charlie	30.0
3	4	NaN	22.0

توضیحات مثال کد پایتون

در این مثال، دو مجموعه داده (**df1** و **df2**) دارای یک کلید مشترک ('ID') هستند. تابع **merge** در pandas برای ادغام این مجموعه داده‌ها بر اساس ستون 'ID' استفاده می‌شود.

یکپارچه‌سازی داده برای ایجاد یک نمای جامع از اطلاعات و روابط در مجموعه داده‌های مختلف ضروری است. در حالی که این مثال بر ادغام مجموعه داده‌ها تمرکز دارد، سایر وظایف یکپارچه‌سازی داده ممکن است شامل مدیریت عدم تطابق شمای داده، حل تعارضات نام‌گذاری و کار با داده‌های از منابع گوناگون باشد.

Section 2.5

2.5.1. Normalization

نرمال‌سازی داده

نرمال‌سازی (Normalization) یک تکنیک پیش‌پردازش داده است که برای مقیاس‌بندی و تبدیل ویژگی‌های عددی در یک مجموعه داده به یک مقیاس مشترک استفاده می‌شود. این فرآیند زمانی حیاتی است که ویژگی‌ها واحدهای یا مقیاس‌های متفاوتی داشته باشند، زیرا تضمین می‌کند که هر ویژگی به طور متناسب در تحلیل مشارکت کند. نرمال‌سازی معمولاً شامل تبدیل داده‌ها به یک مقیاس استاندارد، مانند بین 0 و 1 است.

مثال نرمال‌سازی در پایتون: مقیاس‌بندی Min-Max

توضیح:

مقیاس‌بندی Min-Max یک روش رایج نرمال‌سازی است که مقادیر یک ویژگی را به یک بازه مشخص، معمولاً بین 0 و 1، تبدیل می‌کند. فرمول مقیاس‌بندی Min-Max به صورت زیر است:

$$X_{\text{normalized}} = (X - X_{\text{min}}) / (X_{\text{max}} - X_{\text{min}})$$

```
In [40]: import pandas as pd
from sklearn.preprocessing import MinMaxScaler
import matplotlib.pyplot as plt
import numpy as np
```

این قسمت اول کده. ما سه تا کتابخانه اصلی رو وارد می‌کنیم:
'pandas'، برای کار با داده‌های جدولی
'matplotlib.pyplot' و 'sklearn' از 'sklearn'، برای مقیاس‌گذاری

دارن (Feature) این دیکشنری حاوی داده‌های نمونه ماست که دو تا ویژگی عددی
بین ۵۰۰ تا ۱۵۰۰ 'Feature2' بین ۱۰ تا ۳۰ 'Feature1': این ویژگی‌ها مقیاس‌های متفاوتی دارن
این تفاوت مقیاس می‌تونه بعضی از الگوریتم‌های یادگیری ماشین رو گمراه کنه
data = {
 'Feature1': [10, 20, 15, 25, 30],
 'Feature2': [500, 1000, 750, 1250, 1500]
}

جدید می‌سازیم 'DataFrame' با استفاده از داده‌های بالا، به
df = pd.DataFrame(data)

نرمال‌سازی می‌کنه **Min-Max** این بخش، داده‌ها رو با استفاده از **مقیاس‌گذاری
داده‌ها رو به بازه مشخصی (معمولاً بین ۰ و ۱) منتقل می‌کنه Min-Max مقیاس‌گذاری
این کار باعث می‌شه که همه ویژگی‌ها به یک مقیاس یکسان برسن و هیچ کدوم بر دیگری غلبه نکنه
داده‌ها رو به مدل آموزش می‌ده و همزمان تبدیلیشون می‌کنه 'scaler.fit_transform(df)'
scaler = MinMaxScaler()

```

normalized_data = scaler.fit_transform(df)

# تبدیل می‌کنه تا راحت‌تر قابل نمایش باشن 'DataFrame' این بخش، داده‌های نرمال‌سازی شده رو به په
normalized_df = pd.DataFrame(normalized_data, columns=df.columns)
print("دیتاست نرمال‌سازی شده:")
print(normalized_df)

# رو برای مقایسه داده‌های اصلی و نرمال‌سازی شده رسم می‌کنه (Scatter Plots) این بخش، نمودارهای پراکندگی
# 'plt.subplots()' په فضای رسم با دو تا نمودار در کنار هم ایجاد می‌کنه
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 5))

# نمودار اول: دیتاست اصلی
# 'ax1.scatter()' داده‌های اصلی رو رسم می‌کنه
# 'set_title()' و 'set_xlabel()' و 'set_ylabel()' عنوان و برچسب محورها رو تنظیم می‌کنن
ax1.scatter(df['Feature1'], df['Feature2'], c='blue', label='Original Data')
ax1.set_title('Original Dataset')
ax1.set_xlabel('Feature1')
ax1.set_ylabel('Feature2')
ax1.legend()

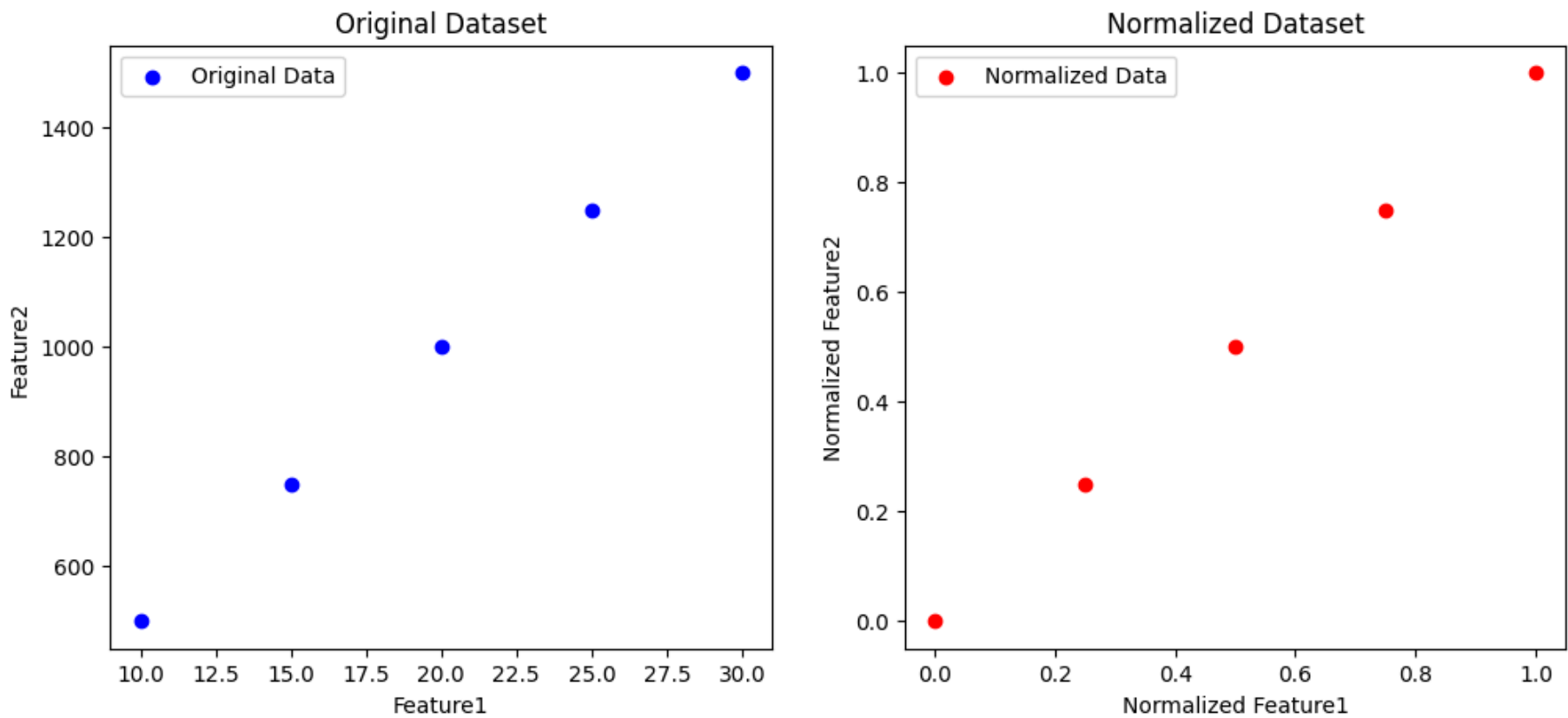
# نمودار دوم: دیتاست نرمال‌سازی شده
# 'ax2.scatter()' داده‌های نرمال‌سازی شده رو رسم می‌کنه
# با نگاه به این نمودار می‌شه دید که داده‌ها فشرده شدن و به بازه کوچیکتری منتقل شدن
ax2.scatter(normalized_df['Feature1'], normalized_df['Feature2'], c='red', label='Normalized Data')
ax2.set_title('Normalized Dataset')
ax2.set_xlabel('Normalized Feature1')
ax2.set_ylabel('Normalized Feature2')
ax2.legend()

# 'plt.show()' نمودارها رو نمایش می‌ده
plt.show()

```

دیتاست نرمال‌سازی شده:

	Feature1	Feature2
0	0.00	0.00
1	0.50	0.50
2	0.25	0.25
3	0.75	0.75
4	1.00	1.00



توضیحات مثال کد پایتون

در این مثال، از `MinMaxScaler` از کتابخانه `scikit-learn` برای نرمال‌سازی ویژگی‌های عددی در مجموعه داده استفاده می‌شود.

نرمال‌سازی برای برخی از الگوریتم‌های یادگیری ماشین، به ویژه آن‌هایی که به مقیاس ویژگی‌های ورودی حساس هستند (مانند `k-nearest neighbors` یا `support vector machines`) بسیار حیاتی است. مثال مقیاس‌بندی `Min-Max` نحوه نرمال‌سازی ویژگی‌های عددی در یک مجموعه داده را نشان می‌دهد.

2.5.2. Discretization



گسسته‌سازی داده (Discretization)

گسسته‌سازی (Discretization) یک تکنیک پیش‌پردازش داده است که شامل تبدیل ویژگی‌های عددی پیوسته به دسته‌ها یا سطل‌های (bins) گسسته می‌شود. این کار در شرایطی که ماهیت پیوسته داده‌ها ضروری نباشد یا هنگام کار با الگوریتم‌هایی که با داده‌های کیفی عملکرد بهتری دارند، مفید است. گسسته‌سازی، داده‌ها را ساده‌سازی کرده و می‌تواند آن‌ها را برای انواع خاصی از تحلیل یا مدل‌سازی مناسب‌تر سازد.

مثال گسسته‌سازی در پایتون: سطل‌بندی با عرض یکسان (Equal Width Binning)

توضیح:

سطل‌بندی با عرض یکسان، یک روش ساده گسسته‌سازی است که بازه مقادیر پیوسته را به فواصل یا سطل‌هایی با عرض برابر تقسیم می‌کند. هر فاصله یک دسته گسسته را نشان می‌دهد و مقادیر در آن بازه به سطل مربوطه اختصاص می‌یابند.

```
In [46]: import pandas as pd
from sklearn.preprocessing import KBinsDiscretizer

# این قسمت اول کده. دو تا کتابخونه رو وارد می‌کنیم
# که برای کار با داده‌ها و جدول‌ها عالیه 'pandas'
# داده‌ها استفاده می‌شه (Discretization) که برای گسسته‌سازی 'sklearn' از 'KBinsDiscretizer'

# این دیکشنری حاوی داده‌های نمونه ماست
# دارن (سن) 'Age' به نام (Continuous) این داده‌ها یک ویژگی عددی پیوسته
data = {
    'Age': [22, 35, 42, 28, 55, 48, 33, 40, 60, 70]
}

# جدید می‌سازیم 'DataFrame' با استفاده از داده‌های بالا، به
df = pd.DataFrame(data)

# تبدیل می‌کنه ** (Equal-Width) با عرض برابر (Binning) این بخش، داده‌ها رو با استفاده از ** گسسته‌سازی
# گسسته‌سازی، داده‌های پیوسته رو به دسته‌های محدود (گسسته) تبدیل می‌کنه
# با عرض برابر تقسیم می‌کنه (bin) این روش، بازه کل داده‌ها رو به تعدادی سطل
# تعداد سطل‌ها رو ۳ تا می‌ذاره 'n_bins=3'
# از روش عرض برابر استفاده می‌کنه 'strategy='uniform''
# هم به هر سطل یه عدد ترتیبی (مثلاً ۰، ۱، ۲) اختصاص می‌ده 'encode='ordinal''
discretizer = KBinsDiscretizer(n_bins=3, encode='ordinal', strategy='uniform', subsample=None)
discretized_data = discretizer.fit_transform(df)

# تبدیل می‌کنیم و چاپشون می‌کنیم 'DataFrame' در نهایت، داده‌های گسسته‌سازی شده رو به یه
print("دیتاست گسسته‌سازی شده")
print(discretized_data)
```

دیتاست گسسته‌سازی شده:

```
[[0.]
 [0.]
 [1.]
 [0.]
 [2.]
 [1.]
 [0.]
 [1.]
 [2.]
 [2.]]
```

توضیحات مثال کد پایتون

در این مثال، از KBinsDiscretizer از کتابخانه scikit-learn برای اعمال سطل‌بندی با عرض یکسان بر روی ویژگی 'Age' استفاده می‌شود که آن را به سه سطل تقسیم می‌کند.

گسسته‌سازی زمانی ارزشمند است که با الگوریتم‌هایی کار می‌کنید که نیاز به ورودی‌های کیفی دارند یا زمانی که می‌خواهید تحلیل داده‌های پیوسته را ساده کنید. مثال سطل‌بندی با عرض یکسان نشان می‌دهد که چگونه می‌توان یک ویژگی عددی را به سطل‌های کیفی گسسته تبدیل کرد.

2.5.3. Data compression

فشرده‌سازی داده (Data Compression)

فشرده‌سازی داده فرآیندی است که برای کاهش حجم یا اندازه داده‌ها به منظور صرفه‌جویی در فضای ذخیره‌سازی و بهبود کارایی محاسباتی استفاده می‌شود. در زمینه داده‌کاوی، تکنیک‌های فشرده‌سازی می‌توانند برای مدیریت کارآمدتر مجموعه‌داده‌های بزرگ، افزایش سرعت پردازش و کاهش نیاز به منابع برای ذخیره‌سازی و انتقال به کار روند.

مثال فشرده‌سازی داده در پایتون: کدگذاری طول اجرا (Run-Length Encoding)

توضیح:

کدگذاری طول اجرا (Run-Length Encoding - RLE) یک تکنیک ساده فشرده‌سازی داده است که مقادیر تکراری متوالی را به عنوان یک مقدار واحد و یک شمارش (count) نمایش می‌دهد. این روش به ویژه برای مجموعه‌داده‌هایی با توالی‌های طولانی از مقادیر یکسان مؤثر است.

```
In [47]: def run_length_encoding(data):
# این تابع، داده‌های تکراری پشت سر هم رو به صورت فشرده درمیاره
# مثلاً داده‌های '1, 1, 1' رو به '1, 3' تبدیل می‌کنه
encoded_data = []
count = 1

# این حلقه روی داده‌ها می‌چرخه تا مقادیر پشت سر هم رو پیدا کنه
for i in range(1, len(data)):
# اگر مقدار فعلی با مقدار قبلی برابر بود، تعداد تکرار رو یکی زیاد می‌کنیم
if data[i] == data[i - 1]:
count += 1
# اگه مقادیر فرق داشتن، تکرار قبلی تموم شده
else:
# اضافه می‌کنیم 'encoded_data' اینجا، داده و تعداد تکرارش رو به لیست
encoded_data.append((data[i - 1], count))
# شمارنده رو برای داده جدید، روی 1 تنظیم می‌کنیم
count = 1

# این خط برای آخرین دسته از داده‌هاست که حلقه تموم شده، ولی هنوز به لیست اضافه نشده
encoded_data.append((data[-1], count))
return encoded_data

# این یه لیست از داده‌های نمونه‌اس که مقادیر تکراری پشت سر هم دارن
data = [1, 1, 1, 2, 3, 3, 3, 3, 4, 4]

# برای فشرده‌سازی داده‌ها استفاده می‌کنیم 'run_length_encoding' اینجا از تابع
compressed_data = run_length_encoding(data)

# در نهایت، داده‌های فشرده‌شده رو چاپ می‌کنیم
print("داده‌های فشرده‌شده:")
print(compressed_data)
```

داده‌های فشرده‌شده:

[(1, 3), (2, 1), (3, 4), (4, 2)]

توضیحات مثال کد پایتون

در این مثال، از تابع `run_length_encoding` برای فشرده‌سازی یک مجموعه داده با مقادیر تکراری متوالی، با استفاده از کدگذاری طول اجرا استفاده می‌شود. فشرده‌سازی داده برای کاهش نیازهای ذخیره‌سازی و افزایش سرعت پردازش داده‌ها مفید است. در حالی که کدگذاری طول اجرا یک مثال پایه است، تکنیک‌های فشرده‌سازی پیشرفته‌تری مانند کدگذاری هافمن (Huffman coding) یا فشرده‌سازی لمپل-زیو (Lempel-Ziv compression) نیز وجود دارند.

2.5.4. Sampling

نمونه‌برداری داده (Sampling)

نمونه‌برداری (Sampling) یک تکنیک پیش‌پردازش داده است که شامل انتخاب یک زیرمجموعه از داده‌ها از یک مجموعه داده بزرگ‌تر می‌شود. این زیرمجموعه که به آن نمونه (sample) گفته می‌شود، نماینده کل جامعه آماری است و اغلب برای اهداف مختلفی مانند تحلیل اکتشافی (exploratory analysis)، آموزش مدل (model training) یا تست عملکرد (performance)

(testing) استفاده می‌شود. نمونه‌برداری به ویژه هنگام کار با مجموعه داده‌های بزرگ ارزشمند است، زیرا امکان پردازش و تحلیل کارآمد یک زیرمجموعه قابل مدیریت را فراهم می‌کند.

مثال نمونه‌برداری در پایتون: نمونه‌برداری تصادفی (Random Sampling)

توضیح:

نمونه‌برداری تصادفی یک تکنیک رایج نمونه‌برداری است که در آن نقاط داده به صورت تصادفی از مجموعه داده انتخاب می‌شوند و یک نمایش بی‌طرفانه از جامعه آماری ارائه می‌دهند.

In [48]:

```
import pandas as pd
import numpy as np

# این قسمت اول کده. ما دو تا کتابخونه رو وارد می‌کنیم
# که برای کار با داده‌های جدولی عالیه 'pandas'
# که برای ایجاد آرایه‌ها و اعداد تصادفی ازش استفاده می‌شه 'numpy'

# این دیکشنری حاوی داده‌های نمونه ماست
# یک لیست از ۱ تا ۱۰۰ رو ایجاد می‌کنه: 'ID'
# صد تا عدد تصادفی بین ۱ تا ۱۰۰ رو ایجاد می‌کنه: 'Value'
# این داده‌ها رو به عنوان یک دیتاست بزرگ در نظر بگیرید که می‌خوایم ازش نمونه‌گیری کنیم
data = {
    'ID': range(1, 101),
    'Value': np.random.randint(1, 100, size=100)
}

# جدید می‌سازیم 'DataFrame' با استفاده از داده‌های بالا، یک
df = pd.DataFrame(data)

# این بخش، **نمونه‌گیری تصادفی** رو انجام می‌ده
# نمونه‌گیری تصادفی، انتخاب یک زیرمجموعه از داده‌ها به صورت کاملاً تصادفیه
# این کار برای تحلیل داده‌های خیلی بزرگ که نمی‌تونیم همه‌شون رو بررسی کنیم، خیلی مفیده
# هست که این کار رو انجام می‌ده 'pandas' تابعی از 'df.sample()'
# به تابع می‌گه که ۲۰ سطر تصادفی رو انتخاب کنه 'n=20'
# هم باعث می‌شه که هر بار که کد رو اجرا می‌کنیم، همون نمونه‌های تصادفی قبلی انتخاب بشن و نتیجه قابل تکرار باشه 'random_state=42'
sample_size = 20
random_sample = df.sample(n=sample_size, random_state=42)

# در نهایت، نمونه تصادفی انتخاب‌شده رو چاپ می‌کنیم
print("داده‌های نمونه‌گیری شده به صورت تصادفی:")
print(random_sample)
```

داده‌های نمونه‌گیری شده به صورت تصادفی:

	ID	Value
83	84	38
53	54	37
70	71	55
45	46	74
44	45	1
39	40	90
22	23	4
80	81	27
10	11	34
0	1	6
18	19	42
30	31	5
73	74	41
33	34	56
90	91	14
4	5	68
76	77	35
77	78	91
12	13	76
31	32	53

توضیحات مثال کد پایتون

در این مثال، از تابع **sample** در **pandas** برای انجام نمونه‌برداری تصادفی از یک مجموعه داده استفاده می‌شود. پارامتر

random_state تکرارپذیری نتایج را تضمین می‌کند.

نمونه‌برداری هنگام کار با مجموعه داده‌های بزرگ برای قابل مدیریت کردن محاسبات و تسریع فرآیند تحلیل ضروری است.

نمونه‌برداری تصادفی تنها یکی از روش‌های متعدد نمونه‌برداری است.

Section 2.6

تحلیل مؤلفه‌های اصلی (PCA)

تحلیل مؤلفه‌های اصلی (Principal Components Analysis - PCA) یک تکنیک کاهش ابعاد (dimensionality reduction)

است که برای تبدیل یک مجموعه داده به فضایی با ابعاد کمتر، در عین حفظ حداکثر واریانس اصلی، استفاده می‌شود. PCA مؤلفه‌های اصلی را شناسایی می‌کند؛ این مؤلفه‌ها بردارهای عمودی هستند که جهت‌های حداکثر واریانس در داده‌ها را به دست می‌آورند. با تصویرسازی داده‌ها روی این مؤلفه‌ها، PCA امکان ارائه یک نمایش خلاصه‌تر از مجموعه داده را فراهم می‌کند که بصری‌سازی و تحلیل آن را آسان‌تر می‌سازد.

مثال تحلیل مؤلفه‌های اصلی در پایتون

بیا یک قطعه کد پایتون با استفاده از کتابخانه‌های **scikit-learn**، **Matplotlib**، **Numpy** و **pandas** برای انجام تحلیل مؤلفه‌های اصلی (PCA) روی یک مجموعه داده نمونه ایجاد کنیم. ما داده‌ها را تولید کرده، PCA را اعمال می‌کنیم و نتایج را بصری‌سازی می‌کنیم.

```
In [50]: import pandas as pd
from sklearn.decomposition import PCA
from sklearn.datasets import load_iris
import matplotlib.pyplot as plt
import numpy as np

# این قسمت اول کده. ما چند تا ابزار مهم رو وارد می‌کنیم
# 'pandas' برای کار با داده‌ها تو قالب جدول
# 'PCA' از 'sklearn' برای کاهش ابعاد استفاده می‌شه
# 'load_iris' از 'sklearn' دیتاست نمونه به اسم
# 'matplotlib.pyplot' برای رسم نمودار

# رو بارگذاری می‌کنه 'iris' این بخش، دیتاست معروف
# هست (iris) این دیتاست شامل ویژگی‌های مختلفی از سه گونه گل زنبق
# می‌داریم 'target' به اسم 'Series' و گونه‌ها رو تو یه 'data' به اسم 'DataFrame' ما ویژگی‌ها رو تو یه
iris = load_iris()
data = pd.DataFrame(data=iris.data, columns=iris.feature_names)
target = pd.Series(data=iris.target, name='target')

# انجام می‌ده PCA رو با استفاده از ** (Dimensionality Reduction) این قسمت، ** کاهش ابعاد
# یه تکنیکه که تعداد ویژگی‌ها رو کم می‌کنه، Principal Component Analysis یا PCA
# در حالی که مهم‌ترین اطلاعات داده‌ها رو نگه می‌داره
# تبدیل کنه (Principal Components) بهش می‌گه که داده‌ها رو به ۲ تا ویژگی اصلی 'n_components=2'
# داده‌ها رو به مدل آموزش می‌ده و تبدیلیشون می‌کنه 'pca.fit_transform(data)'
pca = PCA(n_components=2)
principal_components = pca.fit_transform(data)

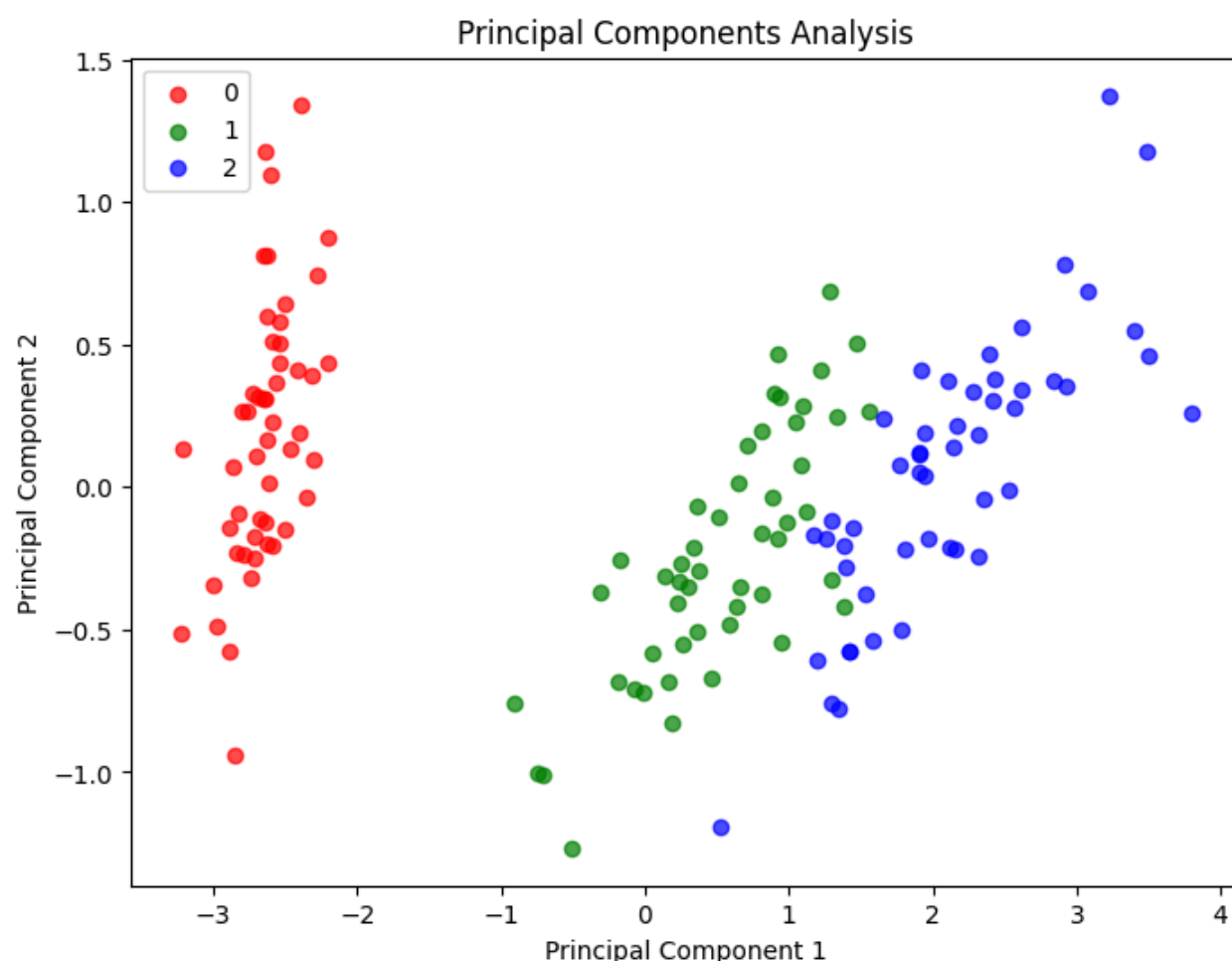
# جدید می‌سازیم که شامل اون دو تا ویژگی اصلی و گونه‌های گل هست 'DataFrame' حالا یه
# این کار برای رسم نمودار بهمون کمک می‌کنه
pc_df = pd.DataFrame(data=principal_components, columns=['PC1', 'PC2'])
pc_df['target'] = target

# رسم می‌کنه (scatter plot) این بخش، داده‌ها رو بعد از کاهش ابعاد، روی یه نمودار پراکندگی
# یه فضای رسم با اندازه مشخص ایجاد می‌کنه 'plt.figure()'
plt.figure(figsize=(8, 6))

# برای رسم هر گونه گل با یک رنگ متفاوت هستن 'colors' و 'targets'
targets = [0, 1, 2]
colors = ['r', 'g', 'b']

# روی گونه‌های مختلف می‌چرخه (for loop) این حلقه
for target, color in zip(targets, colors):
    # این خط، فقط داده‌های مربوط به یک گونه خاص رو انتخاب می‌کنه
    indices_to_keep = pc_df['target'] == target
    # نقاط مربوط به اون گونه رو با رنگ مشخص رسم می‌کنه 'plt.scatter()'
    # به این ترتیب، می‌تونیم ببینیم که گونه‌های مختلف گل چطور در فضای دو بعدی از هم جدا می‌شن
    plt.scatter(pc_df.loc[indices_to_keep, 'PC1'],
                pc_df.loc[indices_to_keep, 'PC2'],
                c=color,
                label=target,
                alpha=0.7)

# این خطوط برای زیباتر شدن و قابل فهم‌تر شدن نمودار هستن
plt.xlabel('Principal Component 1')
plt.ylabel('Principal Component 2')
plt.title('Principal Components Analysis')
plt.legend()
plt.show()
```



توضیحات مثال کد پایتون

در این مثال، **PCA** بر روی مجموعه داده Iris اعمال می‌شود، که شامل اندازه‌گیری‌های طول کاسبرگ، عرض کاسبرگ، طول گلبرگ و عرض گلبرگ برای سه گونه از گل‌های زنبق است. کد ابعاد داده‌ها را به دو مؤلفه اصلی کاهش داده و داده‌ها را در فضای کاهش‌یافته بصری‌سازی می‌کند.

PCA به طور گسترده برای کاهش ویژگی‌ها، کاهش نویز و بصری‌سازی استفاده می‌شود. این مثال نشان می‌دهد که چگونه می‌توان PCA را برای ساده‌سازی و بصری‌سازی داده‌های با ابعاد بالا به کار برد.

2.6.2. Attribute subset selection

انتخاب زیرمجموعه ویژگی‌ها

انتخاب زیرمجموعه ویژگی‌ها (Attribute subset selection) که به آن انتخاب ویژگی (feature selection) نیز گفته می‌شود، شامل شناسایی و انتخاب یک زیرمجموعه از ویژگی‌های مرتبط از مجموعه اصلی صفات در یک مجموعه داده است. هدف این فرآیند بهبود عملکرد مدل، کاهش ابعاد و افزایش قابلیت تفسیرپذیری با حفظ تنها ویژگی‌های آموزنده است. این فرآیند برای مقابله با "نفرین ابعاد (curse of dimensionality)" حیاتی است و می‌تواند به مدل‌های داده‌کاوی کارآمدتر و دقیق‌تر منجر شود.

مثال انتخاب زیرمجموعه ویژگی‌ها در پایتون

بیایید یک قطعه کد پایتون با استفاده از کتابخانه **scikit-learn** برای انجام انتخاب ویژگی (Feature Selection) روی یک مجموعه داده نمونه ایجاد کنیم. ما داده‌ها را بارگذاری کرده، روش‌های مختلف انتخاب ویژگی را بررسی و نتایج را نمایش می‌دهیم.

```
In [53]: import pandas as pd
from sklearn.datasets import load_iris
from sklearn.ensemble import RandomForestClassifier
from sklearn.feature_selection import SelectFromModel
```

```
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
import numpy as np
import matplotlib.pyplot as plt

# وارد می‌کنیم 'sklearn' این قسمت اول کده. ما چند تا ابزار مهم رو از کتابخانه
# 'RandomForestClassifier' ماشینه
# 'SelectFromModel' که برای انتخاب ویژگی‌ها بر اساس اهمیتشون استفاده می‌شه
# 'train_test_split' برای تقسیم داده‌ها به دو بخش آموزش و تست
# 'accuracy_score' برای ارزیابی دقت مدل

# رو بارگذاری می‌کنه 'iris' این بخش، دیتاست معروف
# شامل گونه‌های گل هست 'target' شامل ویژگی‌ها و 'data'
iris = load_iris()
data = pd.DataFrame(data=iris.data, columns=iris.feature_names)
target = pd.Series(data=iris.target, name='target')

# تقسیم می‌کنه (testing) و تست (training) این قسمت، دیتاست رو به دو بخش آموزش
# برای تست مدل استفاده می‌شن 'y_test' و 'X_test' برای آموزش مدل و 'y_train' و 'X_train'
# به این معنی که ۲۰ درصد داده‌ها برای تست استفاده می‌شن 'test_size=0.2'
# هم باعث می‌شه که تقسیم‌بندی هر بار یکسان باشه 'random_state=42'
X_train, X_test, y_train, y_test = train_test_split(data, target, test_size=0.2, random_state=42)

# رو آموزش می‌ده 'RandomForestClassifier' این بخش، یک مدل
# 'RandomForest' یک الگوریتم خیلی قویه که می‌تونه اهمیت هر ویژگی رو برای پیش‌بینی، مشخص کنه
clf = RandomForestClassifier(n_estimators=100, random_state=42)
clf.fit(X_train, y_train)

# استفاده می‌کنیم تا ویژگی‌های مهم رو انتخاب کنیم 'SelectFromModel' حالا از
# بهش می‌گه که فقط ویژگی‌هایی رو انتخاب کنه که اهمیتشون بیشتر از ۰.۲ باشه 'threshold=0.2'
# این کار تعداد ویژگی‌ها رو کم می‌کنه و به مدل کمک می‌کنه که بهتر عمل کنه
sfm = SelectFromModel(clf, threshold=0.2)
sfm.fit(X_train, y_train)

# این بخش، دیتاست‌های آموزش و تست رو بر اساس ویژگی‌های انتخاب‌شده، تغییر می‌ده
# به این ترتیب، فقط با ویژگی‌های مهم کار می‌کنیم
X_train_selected = sfm.transform(X_train)
X_test_selected = sfm.transform(X_test)

# جدید رو با ویژگی‌های انتخاب‌شده آموزش می‌دیم 'RandomForestClassifier' حالا یک مدل
# این مدل فقط از مهم‌ترین ویژگی‌ها برای پیش‌بینی استفاده می‌کنه
clf_selected = RandomForestClassifier(n_estimators=100, random_state=42)
clf_selected.fit(X_train_selected, y_train)
y_pred_selected = clf_selected.predict(X_test_selected)

# مدل رو با مقایسه پیش‌بینی‌ها با مقادیر واقعی محاسبه می‌کنیم (accuracy) در نهایت، دقت
# این مقدار نشون می‌ده که مدل ما چقدر خوب تونسته پیش‌بینی کنه
accuracy = accuracy_score(y_test, y_pred_selected)
print(f'Accuracy with selected features: {accuracy * 100:.2f}%')
```

Accuracy with selected features: 100.00%



توضیحات مثال کد پایتون

در این مثال، از یک **RandomForestClassifier** برای ارزیابی اهمیت ویژگی‌ها استفاده می‌شود و تابع

SelectFromModel برای انتخاب ویژگی‌ها بر اساس اهمیت آن‌ها به کار می‌رود. سپس، طبقه‌بندی‌کننده با استفاده از تنها

ویژگی‌های انتخاب‌شده آموزش داده و ارزیابی می‌شود.

انتخاب زیرمجموعه ویژگی‌ها برای بهبود کارایی مدل، کاهش بیش‌برازش (overfitting) و افزایش قابلیت تفسیرپذیری بسیار

حیاتی است. این مثال نحوه اعمال انتخاب زیرمجموعه ویژگی‌ها را با استفاده از اهمیت ویژگی‌ها در یک طبقه‌بندی‌کننده جنگل

تصادفی نشان می‌دهد.

2.6.3. Nonlinear dimensionality reduction methods



روش‌های کاهش ابعاد غیرخطی

روش‌های کاهش ابعاد غیرخطی (Nonlinear dimensionality reduction methods) با هدف به تصویر کشیدن روابط و

ساختارهای پیچیده در داده‌های با ابعاد بالا، که ممکن است به خوبی توسط تکنیک‌های خطی مانند **PCA** نمایش داده نشوند،

به کار می‌روند. این روش‌ها داده‌ها را به یک فضای با ابعاد کمتر تبدیل می‌کنند، در حالی که روابط غیرخطی ذاتی درون داده‌ها را

حفظ می‌کنند. کاهش ابعاد غیرخطی به ویژه هنگام کار با مجموعه‌داده‌هایی که حاوی الگوهای پیچیده، خوشه‌ها یا منیفلدها

هستند، مفید است.

مثال کاهش ابعاد غیرخطی در پایتون: t-SNE

بیاپید یک قطعه کد پایتون با استفاده از کتابخانه‌های **pandas** و **scikit-learn** برای انجام کاهش ابعاد غیرخطی با استفاده از روش t-Distributed Stochastic Neighbor Embedding (t-SNE) روی یک مجموعه داده نمونه ایجاد کنیم. ما داده‌ها را تولید کرده، t-SNE را اعمال می‌کنیم و نتایج را بصری‌سازی می‌کنیم.

```
In [54]: import pandas as pd
from sklearn.datasets import load_iris
from sklearn.manifold import TSNE
import matplotlib.pyplot as plt
import numpy as np

# این قسمت اول کده. ما چند تا ابزار مهم رو وارد می‌کنیم
# 'pandas' برای کار با داده‌ها تو قالب جدول
# 'TSNE' از 'sklearn' استفاده می‌شه
# 'load_iris' برای وارد کردن یک دیتاست نمونه به اسم
# 'matplotlib.pyplot' برای رسم نمودار

# رو بارگذاری می‌کنه 'iris' این بخش، دیتاست معروف
# هست (iris) این دیتاست شامل ویژگی‌های مختلفی از سه گونه گل زنبق
# می‌داریم 'target' به اسم 'Series' و گونه‌ها رو تو یه 'data' به اسم 'DataFrame' ما ویژگی‌ها رو تو یه
iris = load_iris()
data = pd.DataFrame(data=iris.data, columns=iris.feature_names)
target = pd.Series(data=iris.target, name='target')

# انجام می‌ده t-SNE رو با استفاده از (Nonlinear Dimensionality Reduction) این قسمت، **کاهش ابعاد غیرخطی
# به تکنیک قدرتمنده که داده‌ها رو از فضای با ابعاد بالا به فضای با ابعاد پایین (مثلاً دو بعدی) منتقل می‌کنه
# t-distributed Stochastic Neighbor Embedding، t-SNE با
# t-SNE خوشه‌ها (clusters) برای بصری‌سازی داده‌ها خیلی عالیه، چون الگوها و خوشه‌ها t-SNE
# بهش می‌گه که داده‌ها رو به دو بعد تبدیل کنه 'n_components=2'
# هم باعث می‌شه که نتیجه قابل تکرار باشه 'random_state=42'
tsne = TSNE(n_components=2, random_state=42)
embedded_data = tsne.fit_transform(data)

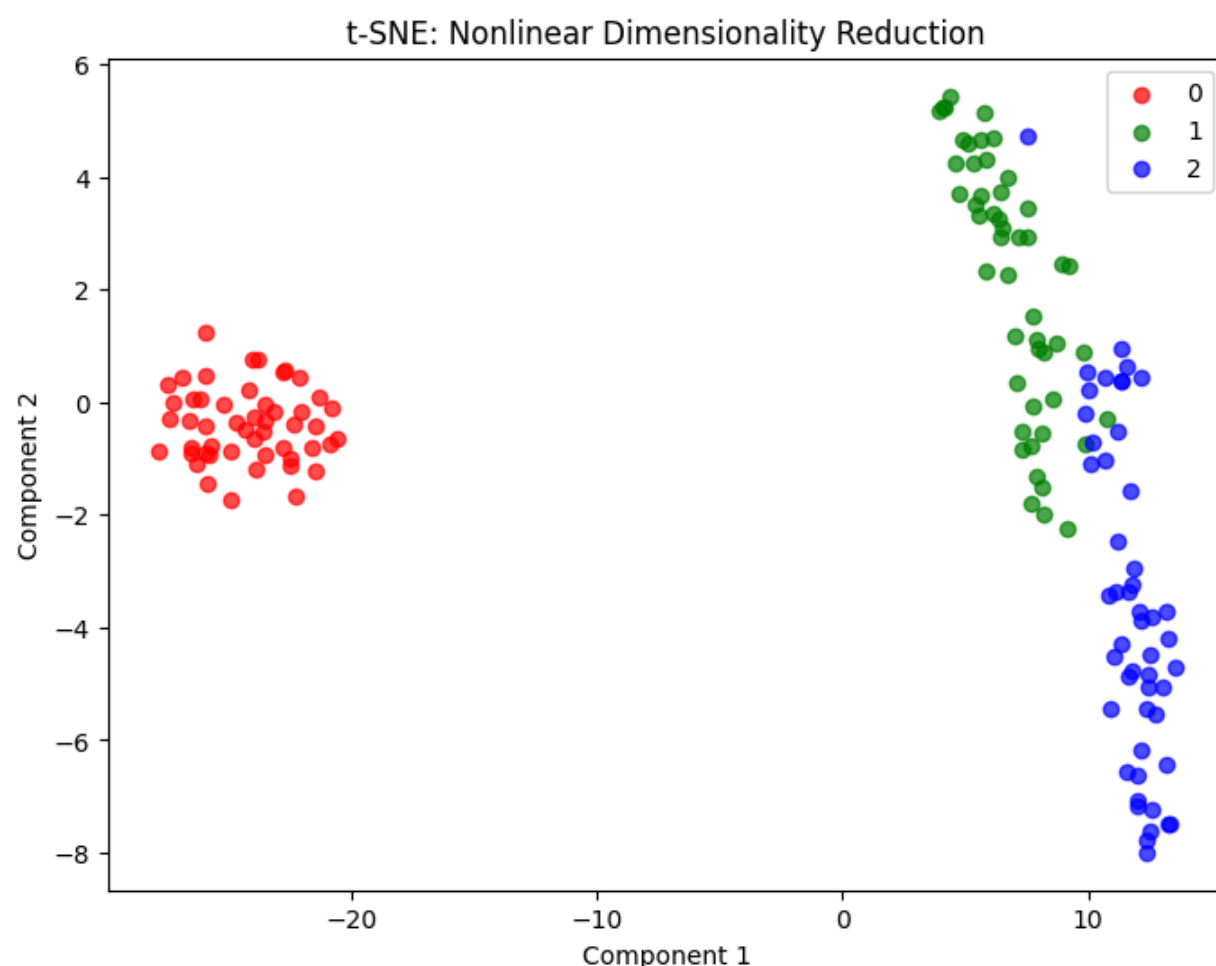
# جدید می‌سازیم که شامل داده‌های تبدیل‌شده و گونه‌های گل هست 'DataFrame' حالا یه
# این کار برای رسم نمودار بهمون کمک می‌کنه
embedded_df = pd.DataFrame(data=embedded_data, columns=['Component 1', 'Component 2'])
embedded_df['target'] = target

# رسم می‌کنه (scatter plot) این بخش، داده‌ها رو بعد از کاهش ابعاد، روی یه نمودار پراکندگی
# یه فضای رسم با اندازه مشخص ایجاد می‌کنه 'plt.figure()'
plt.figure(figsize=(8, 6))

# برای رسم هر گونه گل با یک رنگ متفاوت هستن 'colors' و 'targets'
targets = [0, 1, 2]
colors = ['r', 'g', 'b']

# روی گونه‌های مختلف می‌چرخه (for loop) این حلقه
for target, color in zip(targets, colors):
    # این خط، فقط داده‌های مربوط به یک گونه خاص رو انتخاب می‌کنه
    indices_to_keep = embedded_df['target'] == target
    # نقاط مربوط به اون گونه رو با رنگ مشخص رسم می‌کنه 'plt.scatter()'
    # با نگاه به این نمودار، می‌شه دید که گونه‌های مختلف چطور در فضای دو بعدی از هم جدا شدن
    plt.scatter(embedded_df.loc[indices_to_keep, 'Component 1'],
                embedded_df.loc[indices_to_keep, 'Component 2'],
                c=color,
                label=target,
                alpha=0.7)

# این خطوط برای زیباتر شدن و قابل فهم‌تر شدن نمودار هستن
# عنوان و برچسب محورها به انگلیسی نوشته شدن
plt.xlabel('Component 1')
plt.ylabel('Component 2')
plt.title('t-SNE: Nonlinear Dimensionality Reduction')
plt.legend()
plt.show()
```



توضیحات مثال کد پایتون

در این مثال، t-SNE بر روی مجموعه داده Iris اعمال می‌شود و داده‌ها در یک فضای دوبعدی بصری‌سازی می‌شوند. t-SNE به دلیل حفظ شباهت‌های محلی، برای به تصویر کشیدن ساختارها و الگوهای پیچیده در داده‌ها مؤثر است. روش‌های کاهش ابعاد غیرخطی، مانند t-SNE، برای بصری‌سازی و درک روابط پیچیده در داده‌های با ابعاد بالا ارزشمند هستند. این مثال نحوه استفاده از t-SNE را برای کاهش ابعاد غیرخطی و بصری‌سازی نشان می‌دهد.

The End of Chapter 2